

**République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Carthage
Institut Supérieur des Technologies
de l'Information et de la Communication**

Federated Project Report

Conception and Platform Development "HireFlow"
Quick Wireless Task jobs and Freelance platform

HireFlow

Realized By

Talel Saadani
Aziz Karnit

Supervised by

Dr Wafa Tebourski

Academic year

2025-2026

Table of Contents

1	Requirements Specification.	4
1.1	Introduction	4
1.2	System Context	4
1.3	Identification of Functional Requirements.	4
1.4	Identification of Non-Functional Requirements	5
1.5	Identification of Actors	5
1.6	Global Use Case Diagram	6
1.7	Product Backlog.	7
1.8	Work Environment.	8
1.8.1	Design Methodology	8
1.8.2	Software Environment	9
1.9	Conclusion.	10
2	Sprint 0 – "Authentication & Common Features"	11
2.1	Introduction	11
2.2	Sprint 0 Backlog Identification	11
2.3	Sprint 0 Refinement	12
2.3.1	Refinement of the Use Case "Register"	13
2.3.2	Refinement of the Use Case "Log In"	14
2.3.3	Refinement of the Use Case "Manage Users"	15
2.3.4	Refinement of the Use Case "Manage Jobs"	16
2.3.5	Refinement of the Use Case "Manage Proposals"	17
2.4	Sprint 0 Conception	18
2.4.1	Conception of Use Case "Register"	18
2.4.2	Conception of Use Case "Log In"	20
2.4.3	Conception of Use Case "Manage Users"	21
2.4.4	Conception of Use Case "Manage jobs"	23
2.4.5	Conception of Use Case "Manage proposals"	25
2.5	Deployment Diagram.	27

2.6	Sprint 0 Implementation	28
2.6.1	Registration Interface	28
2.6.2	Login Interface	29
2.6.3	Manage Users Interface	29
2.6.4	Manage Jobs Interface	30
2.6.5	Manage Proposals Interface	30
2.7	Conclusion	31
3	Sprint 1 – "Core Platform Features".	32
3.1	Introduction	32
3.2	Sprint 1 Backlog Identification	32
3.3	Sprint 1 Refinement	32
3.3.1	Refinement of the Use Case “Handle proposals”	33
3.3.2	Refinement of the Use Case “Edit Profile”	34
3.3.3	Refinement of the Use Case “View Dashboard”	35
3.4	Sprint 1 Conception	37
3.4.1	Conception of Use Case “Handle proposals”	37
3.4.2	Conception of Use Case “Edit Profile”	39
3.4.3	Conception of Use Case “View Dashboard”	41
3.5	Sprint 1 Implementation	43
3.5.1	Handle proposals Interface	43
3.5.2	Edit Profile Interface	44
3.5.3	View Dashboard Interface	45
3.6	Conclusion	46
	General Conclusion	47
	Bibliography	48

General Introduction

In recent years, web platforms have become an essential part of the digital economy, enabling individuals and organizations to collaborate, exchange services, and manage professional activities remotely. The rapid evolution of information technologies has significantly transformed the way people work, leading to freelance marketplaces that connect clients with skilled professionals in different domains.

Within the academic environment, students are increasingly seeking opportunities to apply their skills in real projects, collaborate with peers, and gain professional experience before entering the job market. University-oriented freelancing platforms represent an effective solution by providing a structured space where students can offer services, request help, and work on collaborative projects.

In this context, our federated project focuses on the development of a web-based **“Mini Freelancer Marketplace dedicated to university students”** under the name **“HireFlow”**. The platform allows students to act as freelancers by offering their skills while also allowing other students, teams, or clubs to publish job opportunities. Administrators supervise the platform to ensure proper usage and system reliability. The application emphasizes role-based access, workflow automation, and clear interaction between the different actors on the platform.

To model and implement this system, we adopted an object-oriented design approach following an iterative and incremental methodology based on **“SCRUM”**. This methodology allows progressive development of the platform through multiple sprints, ensuring continuous improvement and validation of functional requirements.

This report is organized into three main chapters:

- The first chapter, entitled ***"Requirements Specification"***, presents the system context, functional and non-functional requirements, identification of actors, the global use case diagram, and the product backlog.
- The second chapter introduces the initial sprint, including use case refinement, system design, and the first implementation steps.
- The third chapter focuses on advanced implementation aspects, system integration, testing, and final improvements.

Finally, we conclude this report with a general conclusion highlighting the achieved results and presenting future perspectives for enhancing the platform.

1 Requirements Specification

1.1 Introduction

Requirements analysis is an essential step for the success of a system. It consists of identifying user expectations for an application under development in order to better understand their needs and define the functionalities to be implemented. This chapter presents the identification of functional and non-functional requirements of the system as well as the general context of the platform. It also introduces the main actors and the foundations necessary for the system design.

1.2 System Context

With the evolution of digital technologies in education, students increasingly seek opportunities to apply their skills to real projects and collaborate with their peers. However, existing freelancing platforms are not specifically adapted to academic environments where the primary objective is skill development.

The proposed system is a web-based **"Mini Freelancer Marketplace dedicated to university students"**. It enables students to offer their skills as freelancers, while other students, teams, or university clubs can publish project opportunities. The platform also includes administrative supervision to ensure proper usage and overall system management.

1.3 Identification of Functional Requirements

A functional requirement specifies an action that the system must be able to perform from the user's perspective. Functional requirements describe the services provided by the platform and define interactions between system actors.

Our system allows users to:

- Register;
- log in;
- Manage proposals;
- Handle proposals;
- Manage users;
- Manage jobs;
- Edit Profile;
- View dashboard.

1.4 Identification of Non-Functional Requirements

A non-functional requirement specifies system properties and constraints such as performance, security, maintainability, and usability. These requirements ensure the overall quality of the application.

Our system must satisfy the following non-functional requirements:

- **Usability:** the application must provide a simple and intuitive interface suitable for students.
- **Security:** the system must ensure user authentication and data protection.
- **Robustness:** the application must operate correctly despite errors or interruptions.
- **Maintainability:** the code must be structured and extensible for future improvements.
- **Performance:** the platform must guarantee fast response times.

These requirements ensure the reliability and quality of the platform within an academic environment.

1.5 Identification of Actors

The Mini Freelancer Marketplace involves several actors who interact with the system according to their roles and permissions. Identifying these actors helps clarify responsibilities and system boundaries.

The actors of the system is shown in Figure 1.

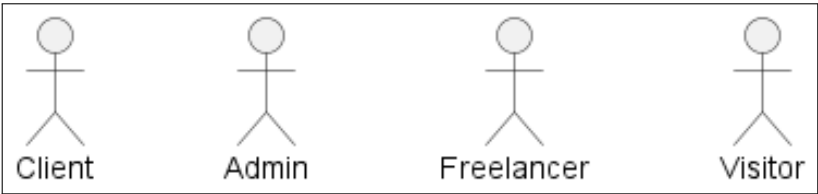


Figure 1: Actors

Actor	Description
Visitor (Guest)	An unregistered user who can view public information and access registration or login pages.
User	A registered user who can log in, edit their profile (update personal information, change password, upload profile picture), and view a dashboard summarizing their activity.
Client	A user who can create, manage, and monitor job postings, view received proposals, and accept or reject freelancer applications.
Freelancer	A user who can browse available jobs and create, delete, or submit proposals to clients.
Administrator	A privileged user who can manage users, validate and monitor jobs, and view system statistics.

Table 1: Actors Description

1.6 Global Use Case Diagram

“The global use case diagram provides a high-level view of the interactions between system actors and the Mini Freelancer Marketplace. It summarizes the main services offered by the platform such as authentication, job management, proposal submission, and administrative supervision.” — Object Management Group (OMG)

The global use case diagram of the system is shown in Figure 2.

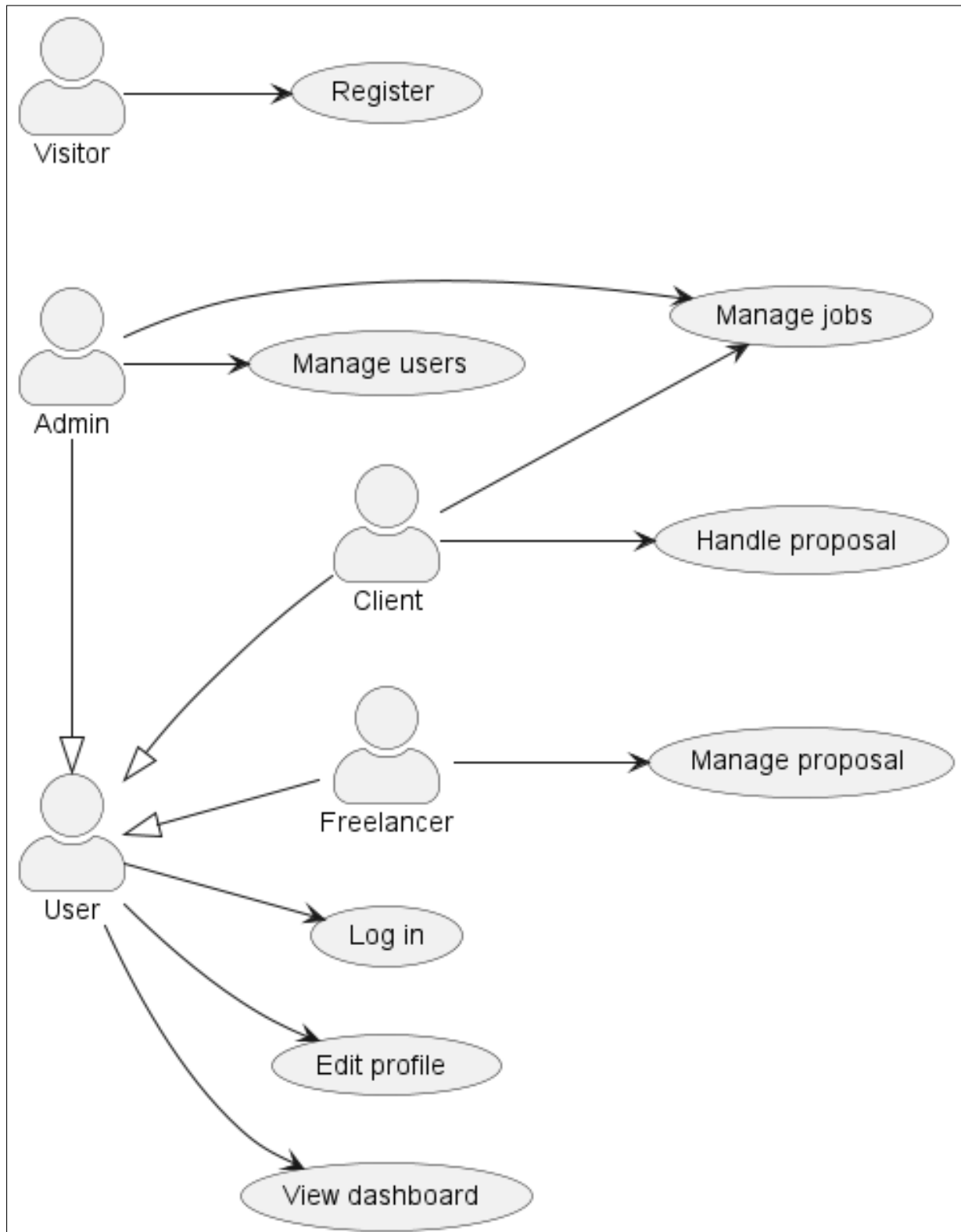


Figure 2: Global Use Case Diagram

1.7 Product Backlog

“The product backlog is an ordered list of system functionalities expressed as user stories. It serves as the primary planning artifact in the SCRUM methodology and evolves throughout the project life-cycle.”

— [[2]] [The Scrum Guide].

ID	User Story	Priority
US1	As a visitor, I want to register so that I can create an account.	1
US2	As a user, I want to log in so that I can access my account.	1
US3	As a freelancer, I want to manage my proposals so that I can track, or withdraw them.	1
US4	As an administrator, I want to manage users so that I can control access and maintain the system.	1
US5	As an administrator or client, I want to manage jobs so that I can create or delete job postings.	1
US6	As a client, I want to handle proposals so that I can accept or reject applications.	2
US7	As a user, I want to view a dashboard so that I can monitor my activity and data.	2
US8	As a user, I want to edit my profile so that I can update my personal information.	2

Table 2: Product Backlog

1.8 Work Environment

This section describes the methodological and technical environment adopted to design and implement the **"Mini Freelancer Marketplace - HireFlow -"**.

1.8.1 Design Methodology

The development of this project follows the **"SCRUM"** agile methodology.

“**SCRUM** is based on iterative and incremental development organized into short time-boxed iterations called sprints.” —[\[SCRUM\]](#) [\[\[1\]\]](#)

The adoption of **"SCRUM"** provides several advantages:

- Continuous delivery of functional increments;
- Better adaptability to changes in requirements;
- Improved collaboration and project visibility;
- Early detection of risks and issues.

The project **"Workflow"** follows these steps:

- Product backlog preparation;
- Sprint planning;
- Sprint execution;
- Sprint review and retrospective.

This approach ensures progressive validation of the platform functionalities.

1.8.2 Software Environment

“HireFlow” is developed using a modern full-stack web architecture.

Table 3: Complete Technologies Stack Overview

Technology	Definition
Frontend Technologies	
 Angular	Angular is a modern open-source front-end framework developed by Google. It is used to build dynamic and scalable single-page applications using a component-based architecture and TypeScript. [[3]]
 TypeScript	TypeScript is a strongly-typed superset of JavaScript developed by Microsoft. It adds static typing and object-oriented features to JavaScript, improving code reliability and maintainability in large-scale applications. [[4]]
 HTML5 & CSS3	HTML5 and CSS3 are the standard technologies for structuring and styling web pages. HTML5 defines the semantic content of web documents, while CSS3 handles layout, animations, and visual presentation.
Backend Technologies	
 FastAPI	FastAPI is a high-performance Python web framework for building APIs. It provides automatic data validation, serialization, and interactive documentation via OpenAPI, making it one of the fastest frameworks available for Python. [[5]]
 Python	Python is a versatile, easy-to-read, and widely-used programming language. Its rich ecosystem of libraries and frameworks makes it a top choice for back-end development, data processing, and automation. [[6]]
Database	
 MySQL	MySQL is a widely used open-source relational database management system known for its performance, ease of use, and scalability. It supports standard SQL queries, transactions, and is commonly used in web applications. [[7]]

Technology	Definition
Development Tools	
 Visual Studio Code	Visual Studio Code is a lightweight but powerful open-source code editor developed by Microsoft. It runs on Windows, macOS, and Linux, and provides developers with an integrated development environment featuring rich extensions, debugging tools, and Git integration. [[8]]
 Git & GitHub	Git is a distributed version control system. A local clone of a project is a complete version-control repository, facilitating offline or remote work. Developers validate their work locally, then synchronize their copy with the remote server. GitHub extends Git with collaborative features such as pull requests, issues, and project boards. [[9]]
 StarUML / PlantUML	StarUML and PlantUML are modeling tools used to design UML diagrams and visualize system architecture. They support use case diagrams, class diagrams, sequence diagrams, and deployment diagrams, making them well-suited for software documentation. [[11]] [[12]]

This technological stack ensures scalability, maintainability, and high performance of the platform.

1.9 Conclusion

In this chapter, we identified the main actors of “**HireFlow**” and presented a global vision of the system through the use case perspective. We also established the product backlog that guides the iterative development process. Finally, we described the methodological and software environment adopted for the project. These elements constitute the foundation for the design and implementation phases that will be detailed in the next chapters.

2 Sprint 0 – "Authentication & Common Features"

2.1 Introduction

In this chapter, we present the first sprint of the project, which details the priority use cases. The study of this sprint covers the **refinement**, **conception**, and **implementation**.

Sprint 0 focuses on the authentication system and the common features shared by all users of the platform. These are the foundational elements on which all subsequent sprints will build. At the end of this sprint, every user – "**Client, Freelancer, or Administrator**" – must be able to **register, log in, manage** their profile, and **access** a role-specific dashboard.

2.2 Sprint 0 Backlog Identification

The sprint backlog contains all *User Stories* selected from the **Product Backlog** for this sprint. Each User Story is derived directly from the use case diagram and follows the "**INVEST**" criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable).

ID	User Story	Estimation
US01	As a visitor, I want to register so that I can create an account.	High
US02	As a user, I want to log in so that I can access my account.	Medium
US03	As a freelancer, I want to manage my proposals (submit, withdraw) so that I can track my applications.	Low
US04	As an administrator, I want to manage users (view, delete) so that I can maintain system control.	Medium
US05	As a client, I want to manage jobs (create, delete) so that I can control job postings.	Medium

Table 4: Sprint 0 Backlog - Core Freelance Functionalities

A User Story is considered "**completed**" when:

- The code is developed and reviewed, unit tests pass successfully, the functionality is validated by the Product Owner, and the technical documentation is up to date.

2.3 Sprint 0 Refinement

In this section, we focus on the following use cases:

- Register;
- log in;
- Manage users;
- Manage jobs;
- Manage proposals.

2.3.1 Refinement of the Use Case “Register”

The diagram below illustrates the Register use case and its interactions with the system actors.

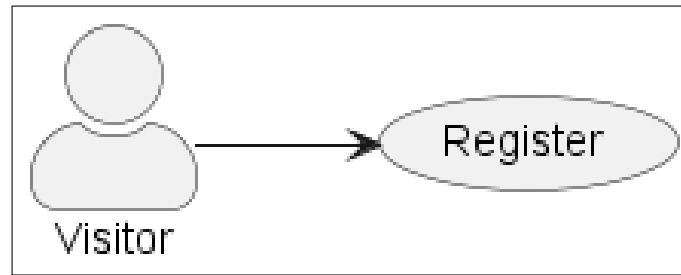


Figure 3: Refinement of the use case “Register”

Table 5 presents the textual description of the use case “Register”:

Use Case	Register
Actor	Visitor (unregistered user)
Pre-condition	The user is not yet registered on the platform
Post-condition	A new user account is created; the user is redirected to the login page
Main Scenario	1. The visitor navigates to the registration page. 2. The system displays the registration form (name, email, password, role). 3. The visitor fills in all required fields and selects a role (Client / Freelancer). 4. The visitor submits the form. 5. The system validates the data (format checks, unique email). 6. The system hashes the password and saves the new user to the database. 7. The system confirms successful registration.
Alternative Scenarios	• Email already exists: the system displays an error message. • Invalid format (e.g., malformed email): the system highlights the field. • Missing required field: the system blocks submission and indicates the field.
Exception	Database connection failure; the system displays a generic error.

Table 5: Textual description — Register

2.3.2 Refinement of the Use Case “Log In”

The diagram below illustrates the "Log In" use case and its interactions with the system actors.

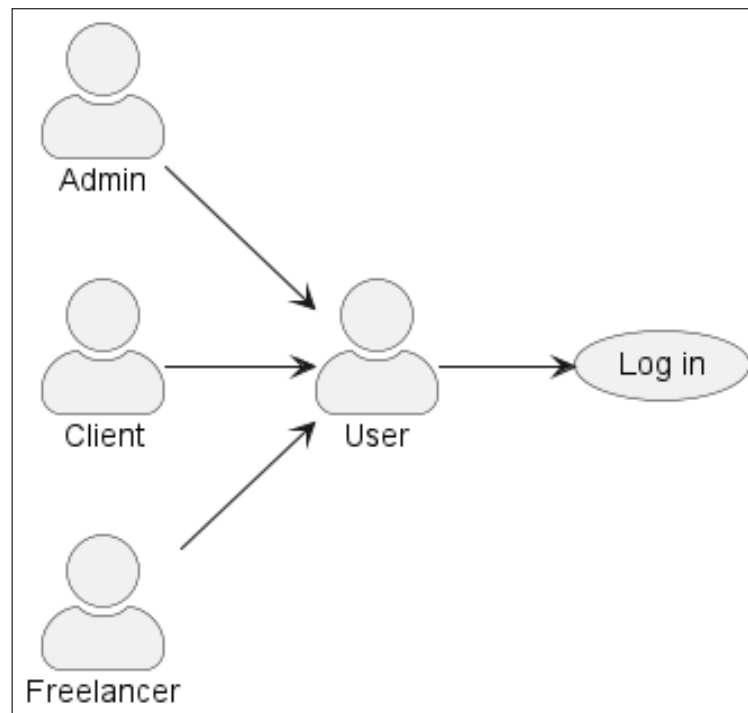


Figure 4: Refinement of the use case “Log In”

Table 6 presents the textual description of the use case “Log In”:

Use Case	Log In
Actor	Registered User (Client / Freelancer / Admin)
Pre-condition	The user has a registered account on the platform
Post-condition	The user is authenticated and redirected to their role-specific dashboard
Main Scenario	1. The user navigates to the login page. 2. The system displays the login form (email, password). 3. The user enters their credentials and submits. 4. The system verifies the email and password from the database. 5. The system generates a JWT (valid for 1 hour) and sends it to the client. 6. The user is redirected to their dashboard.
Alternative Scenarios	• Incorrect password: the system returns a 401 error with an appropriate message. • Unregistered email: the system displays an account-not-found message. • Account suspended: the system informs the user to contact the administrator.
Exception	Database unavailable; the system returns a 500 error.

Table 6: Textual description — Log In

2.3.3 Refinement of the Use Case “Manage Users”

The diagram below illustrates the "Manage Users" use case and its interactions with the system actors.

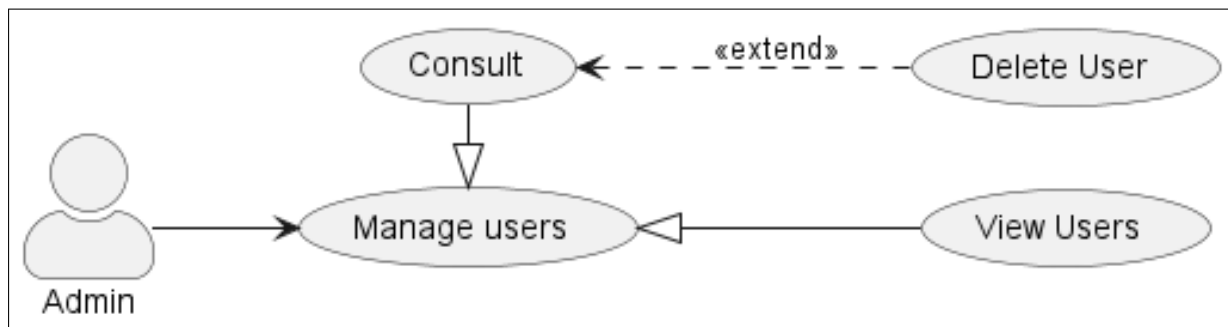


Figure 5: Refinement of the use case “Manage Users”

Table 7 presents the textual description of the use case “Manage Users”:

Use Case	Manage Users
Actor	Administrator
Pre-condition	The administrator is logged in with appropriate privileges
Post-condition	User data is updated or deleted successfully in the system
Main Scenario	1. The administrator navigates to the user management interface. 2. The system displays the list of users. 3. The administrator selects a user. 4. The system displays the user’s details. 5. The administrator chooses to delete. 6. The system validates the requested action. 7. The system deletes the user in the database. 8. The system confirms the successful operation.
Alternative Scenarios	• Invalid update data: the system displays validation errors and requests correction. • Deletion not allowed: the system blocks deletion if the user is linked to important data.
Exception	Database connection failure; the system displays an error message.

Table 7: Textual description — Manage Users

2.3.4 Refinement of the Use Case “Manage Jobs”

The diagram below illustrates the "Manage Jobs" use case and its interactions with the system actors.

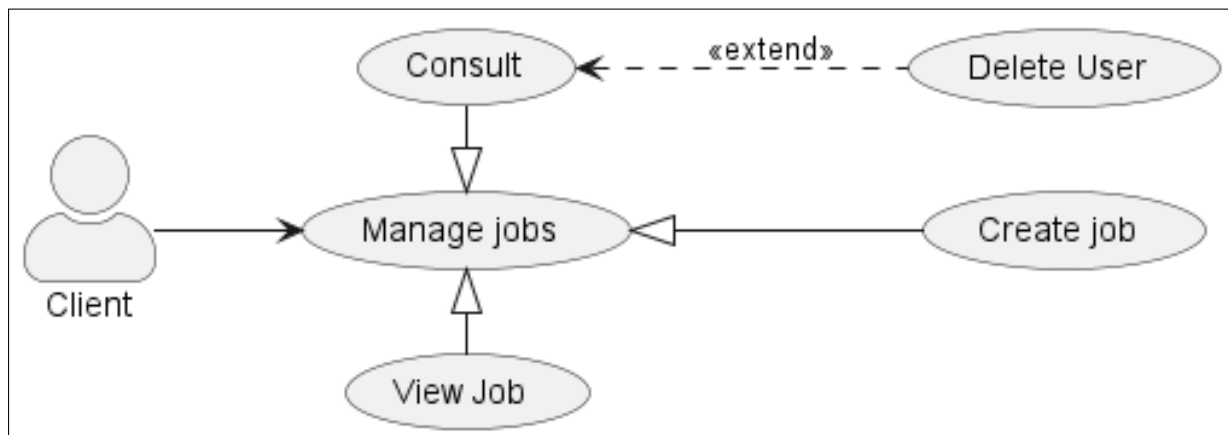


Figure 6: Refinement of the use case “Manage Jobs”

Table 8 presents the textual description of the use case “Manage Jobs”:

Use Case	Manage Jobs
Actor	Client / Administrator
Pre-condition	The user is logged in with a Client or Administrator role
Post-condition	Job data is created, updated, or deleted successfully
Main Scenario	1. The client navigates to the job management interface. 2. The system displays the list of jobs. 3. The client selects an action (create or delete a job). 4. If creating, the system displays the job creation form. 5. The client fills in the required fields and submits. 6. The system validates the data. 7. The system saves the job in the database, or deletes it. 8. The system confirms the successful operation.
Alternative Scenarios	• Missing or invalid data: the system displays validation errors. • Deletion canceled: the client aborts the delete operation.
Exception	Database connection failure; the system displays an error message.

Table 8: Textual description — Manage Jobs

2.3.5 Refinement of the Use Case “Manage Proposals”

The diagram below illustrates the "**Manage Proposals**" use case and its interactions with the system actors.

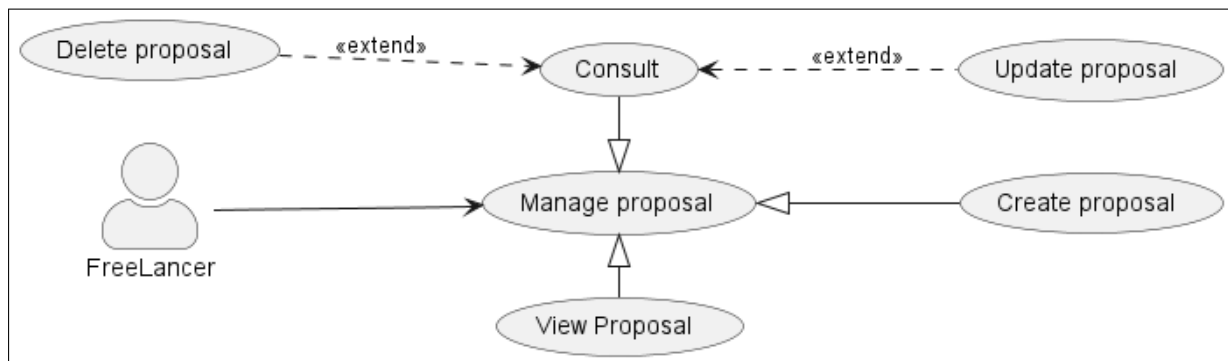


Figure 7: Refinement of the use case “Manage Proposals”

Table 9 presents the textual description of the use case “Manage Proposals”:

Use Case	Manage Proposals
Actor	Freelancer
Pre-condition	The user is logged in with a Freelancer role
Post-condition	Proposal is created, updated, or withdrawn successfully
Main Scenario	1. The freelancer navigates to the jobs section. 2. The system displays the list of available jobs. 3. The freelancer selects a specific job. 4. The system displays the job details. 5. The freelancer chooses to submit a proposal. 6. The system displays the proposal submission form. 7. The freelancer fills in the required details and submits. 8. The system validates the data. 9. The system saves the proposal in the database and confirms the operation. 10. The freelancer navigates to the proposals section. 11. The system displays the list of the freelancer’s proposals. 12. The freelancer selects a proposal. 13. The freelancer chooses to update or delete the proposal. 14. The system validates the action and updates or removes the proposal. 15. The system confirms the successful operation.
Alternative Scenarios	• Missing or invalid data: the system displays validation errors. • Withdrawal canceled: the freelancer aborts the withdrawal action.
Exception	Database connection failure; the system displays an error message.

Table 9: Textual description — Manage Proposals

2.4 Sprint 0 Conception

For each Use Case model, we present the Class diagram and the Sequence diagram.

2.4.1 Conception of Use Case “Register”

a) Class Diagram — Register

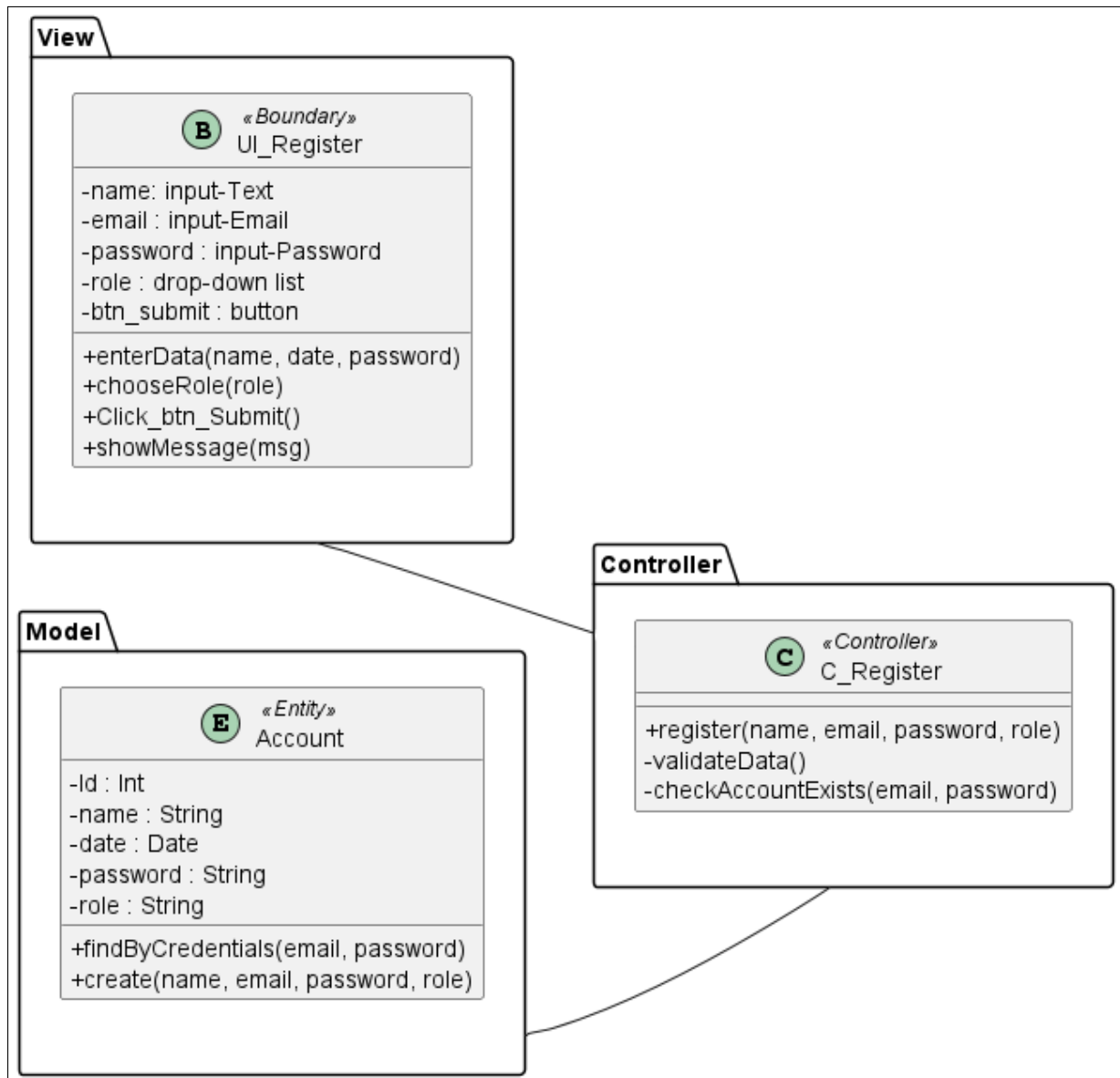


Figure 8: Class diagram — Register

b) Sequence Diagram — Register

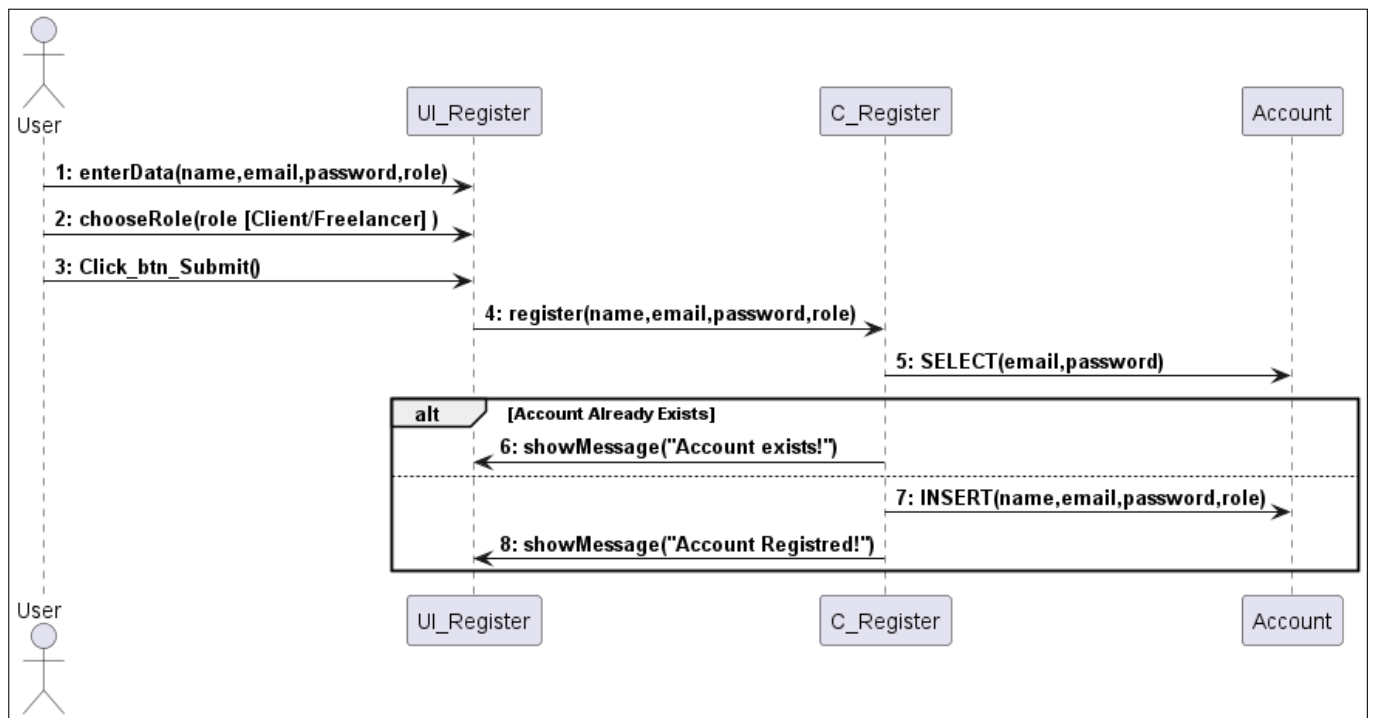


Figure 9: Sequence diagram — Register

2.4.2 Conception of Use Case “Log In”

a) Class Diagram — Log In

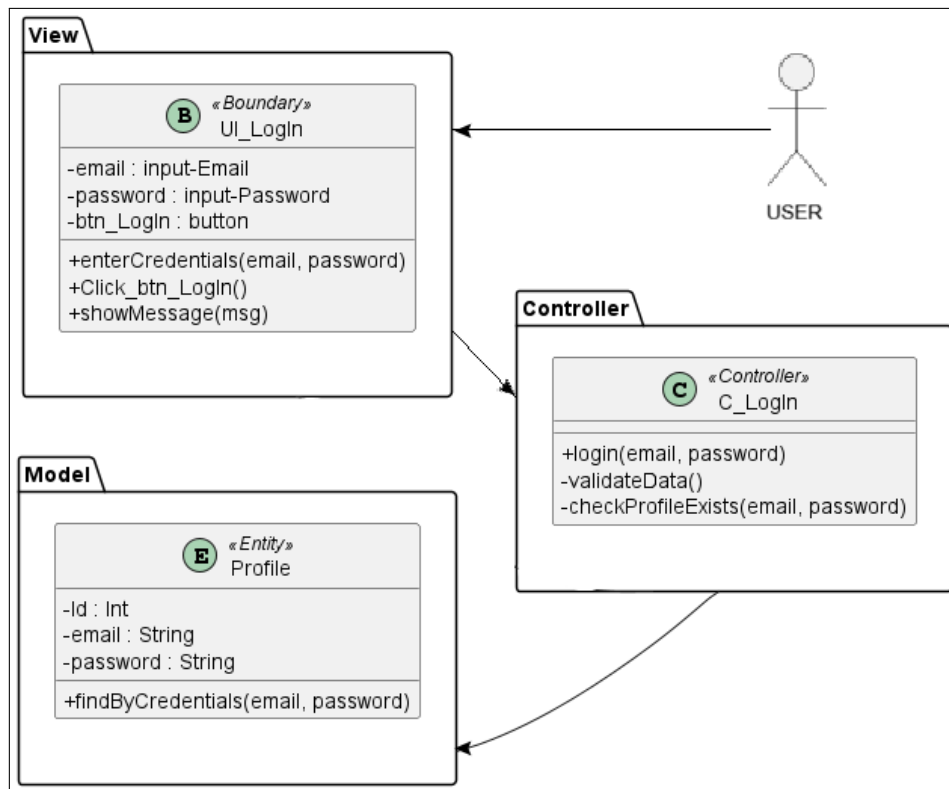


Figure 10: Class diagram — Log In

b) Sequence Diagram — Log In

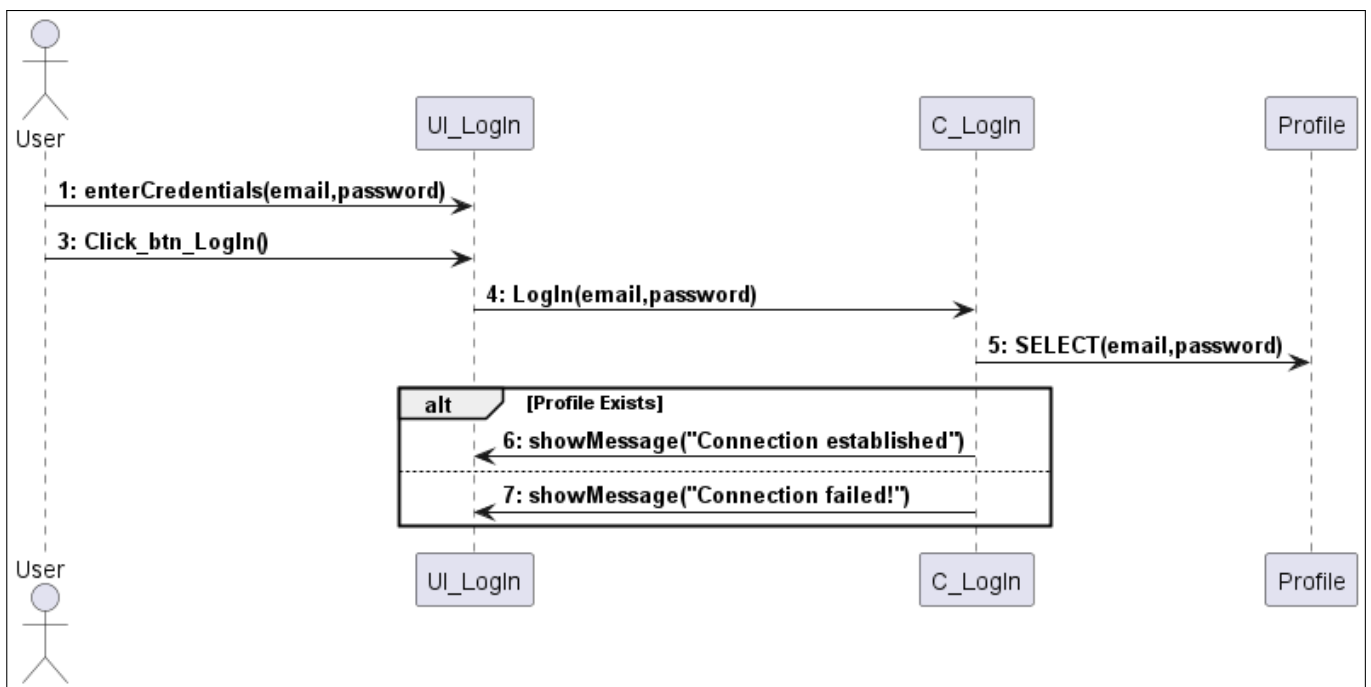


Figure 11: Sequence diagram — Log In

2.4.3 Conception of Use Case “Manage Users”

a) Class Diagram — Manage Users

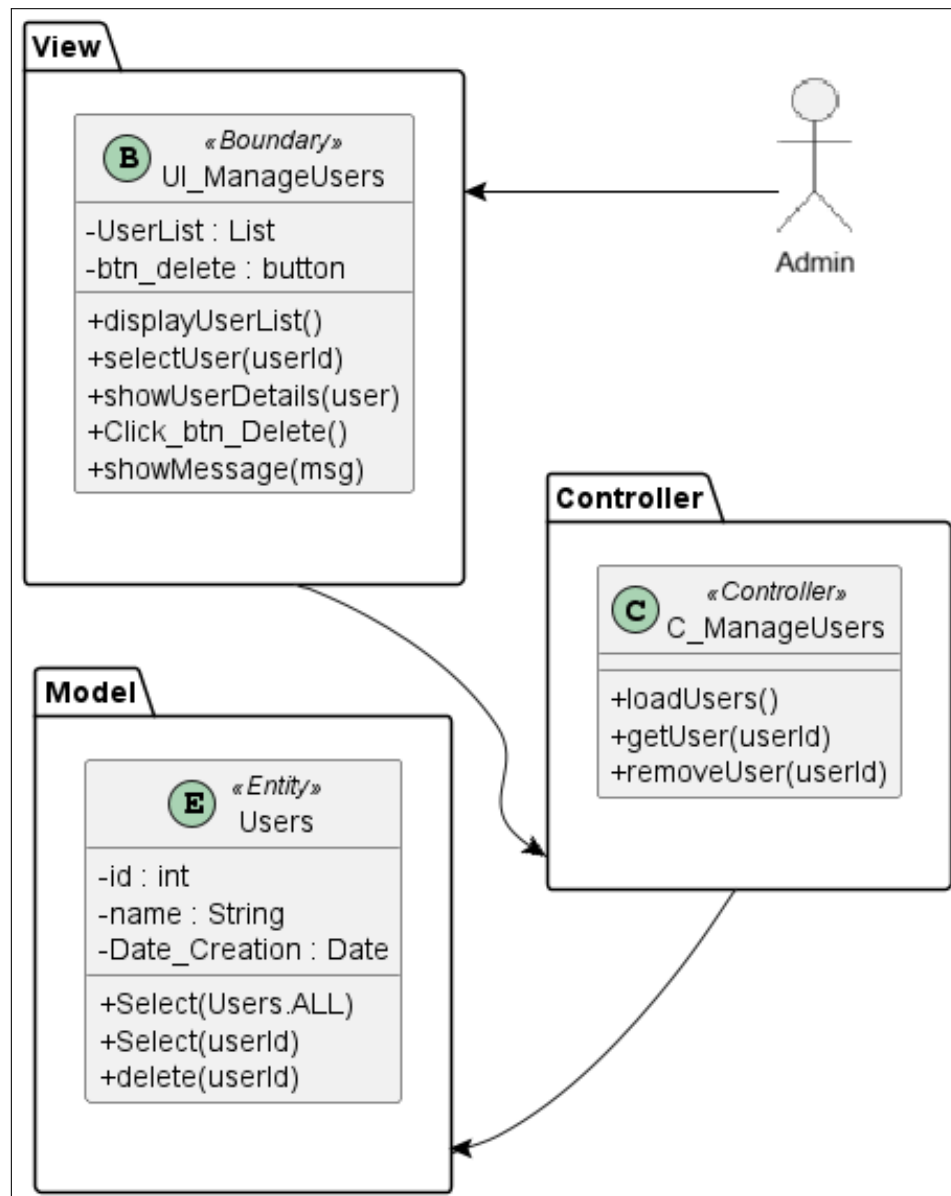


Figure 12: Class diagram — Manage Users

b) Sequence Diagram — Manage Users

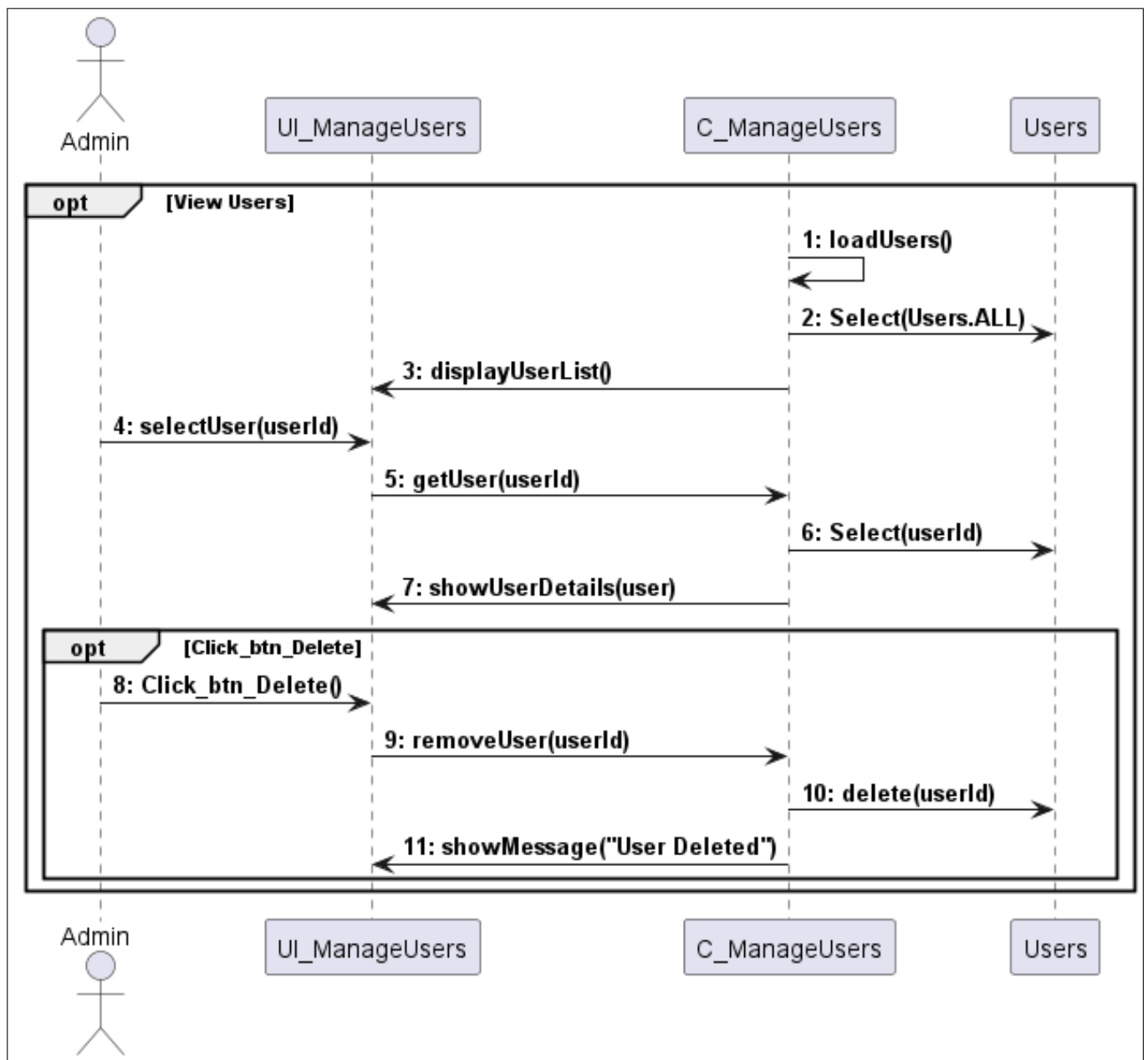


Figure 13: Sequence diagram — Manage Users

2.4.4 Conception of Use Case “Manage jobs”

a) Class Diagram — Manage jobs

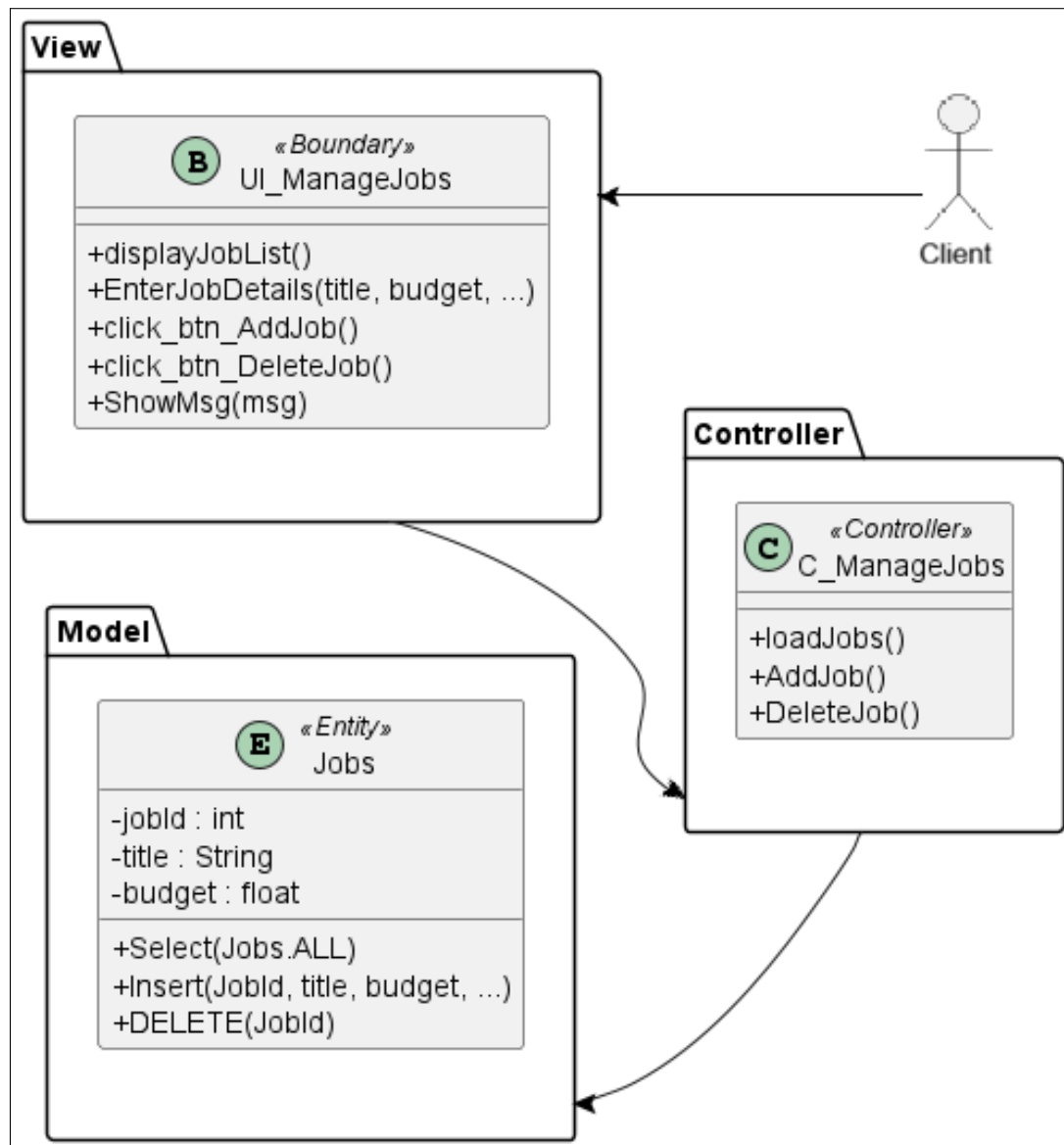


Figure 14: Class diagram — Manage Jobs

b) Sequence Diagram — Manage Jobs

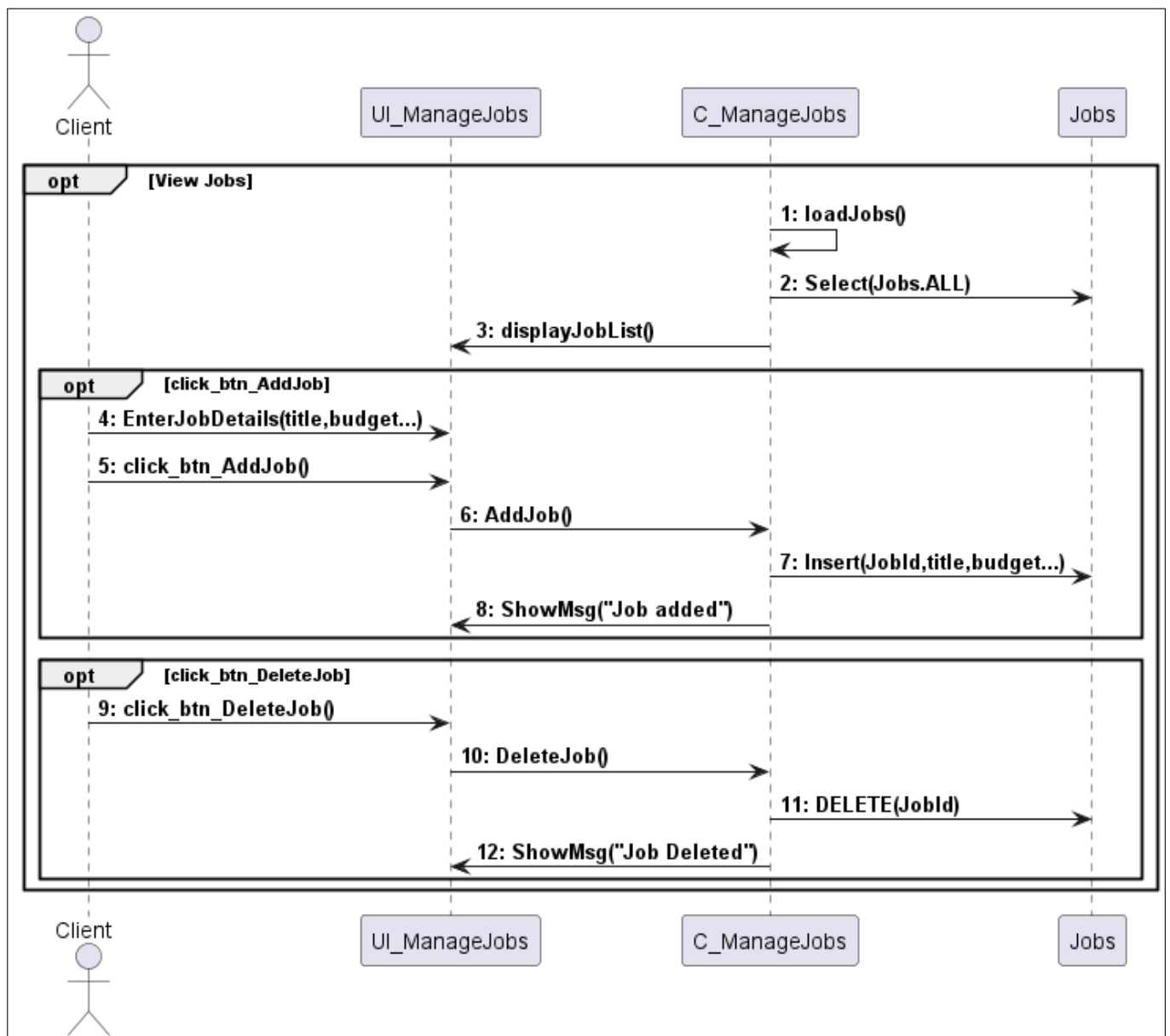


Figure 15: Sequence diagram — Manage Jobs

2.4.5 Conception of Use Case “Manage proposals”

a) Class Diagram — Manage proposals

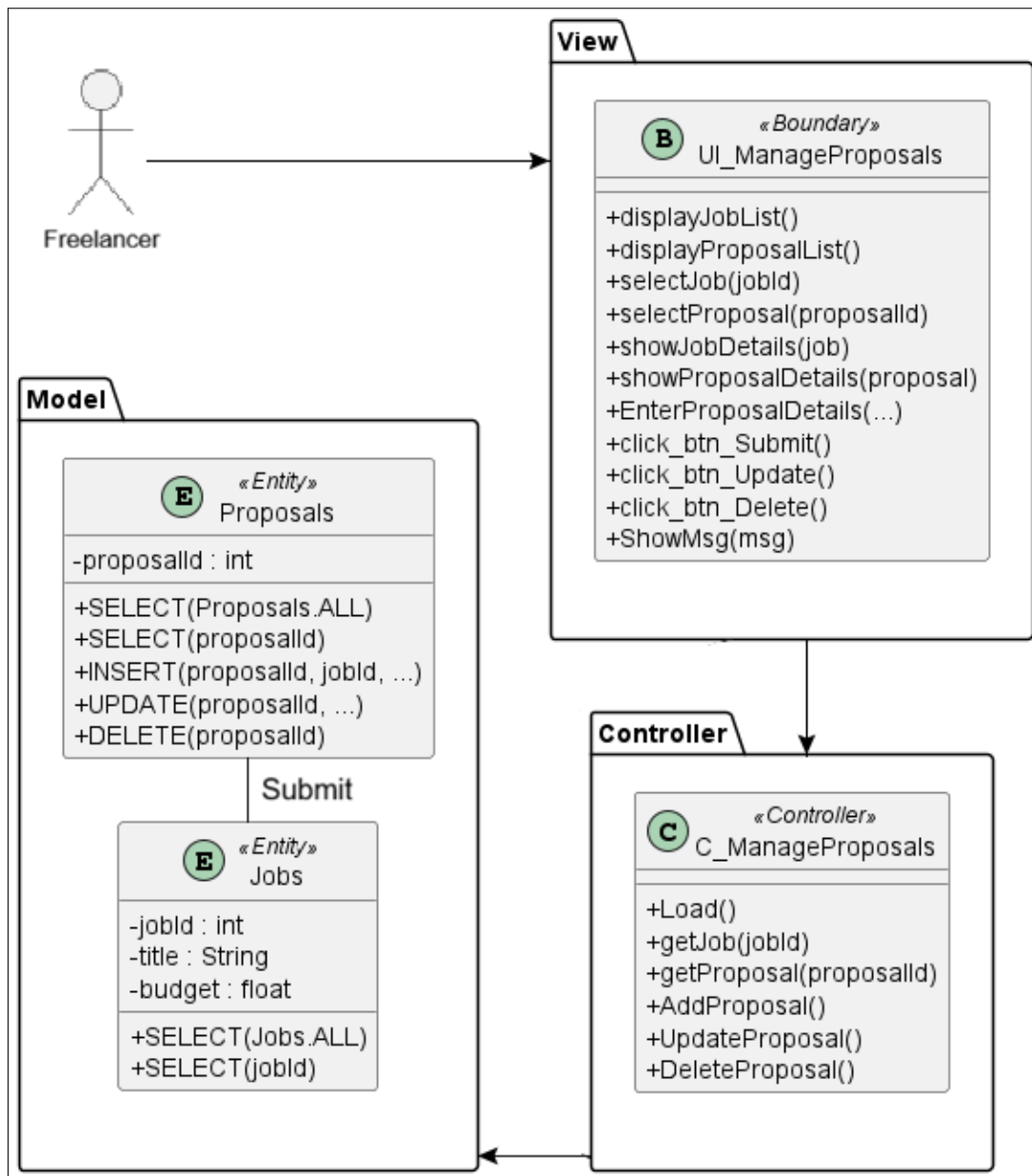


Figure 16: Class diagram — Manage Proposals

b) Sequence Diagram — Manage Proposals

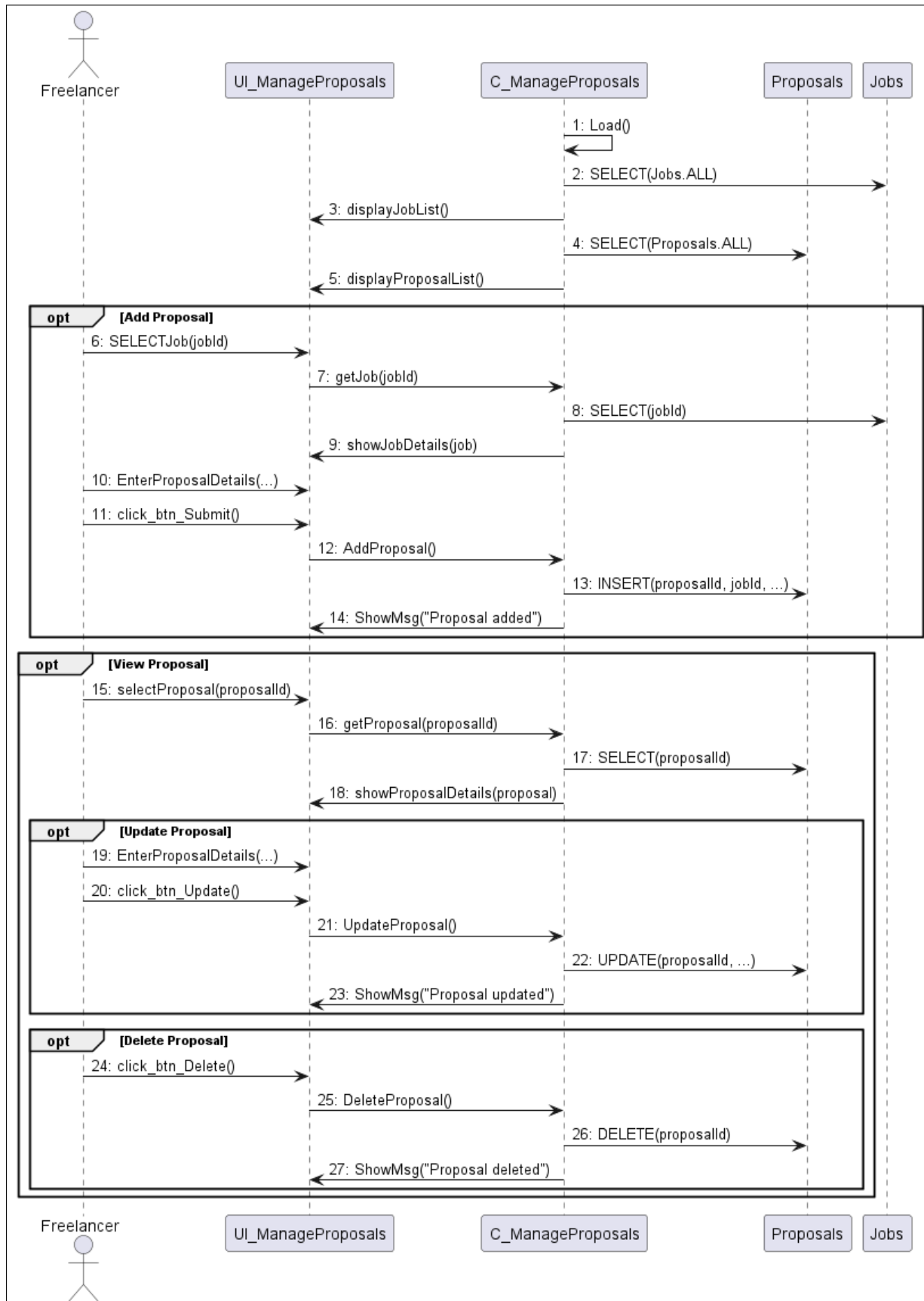


Figure 17: Sequence diagram — Manage Proposals

2.5 Deployment Diagram

The deployment diagram below illustrates the physical architecture of the platform, showing how the different components (frontend, backend, database) are distributed across nodes.

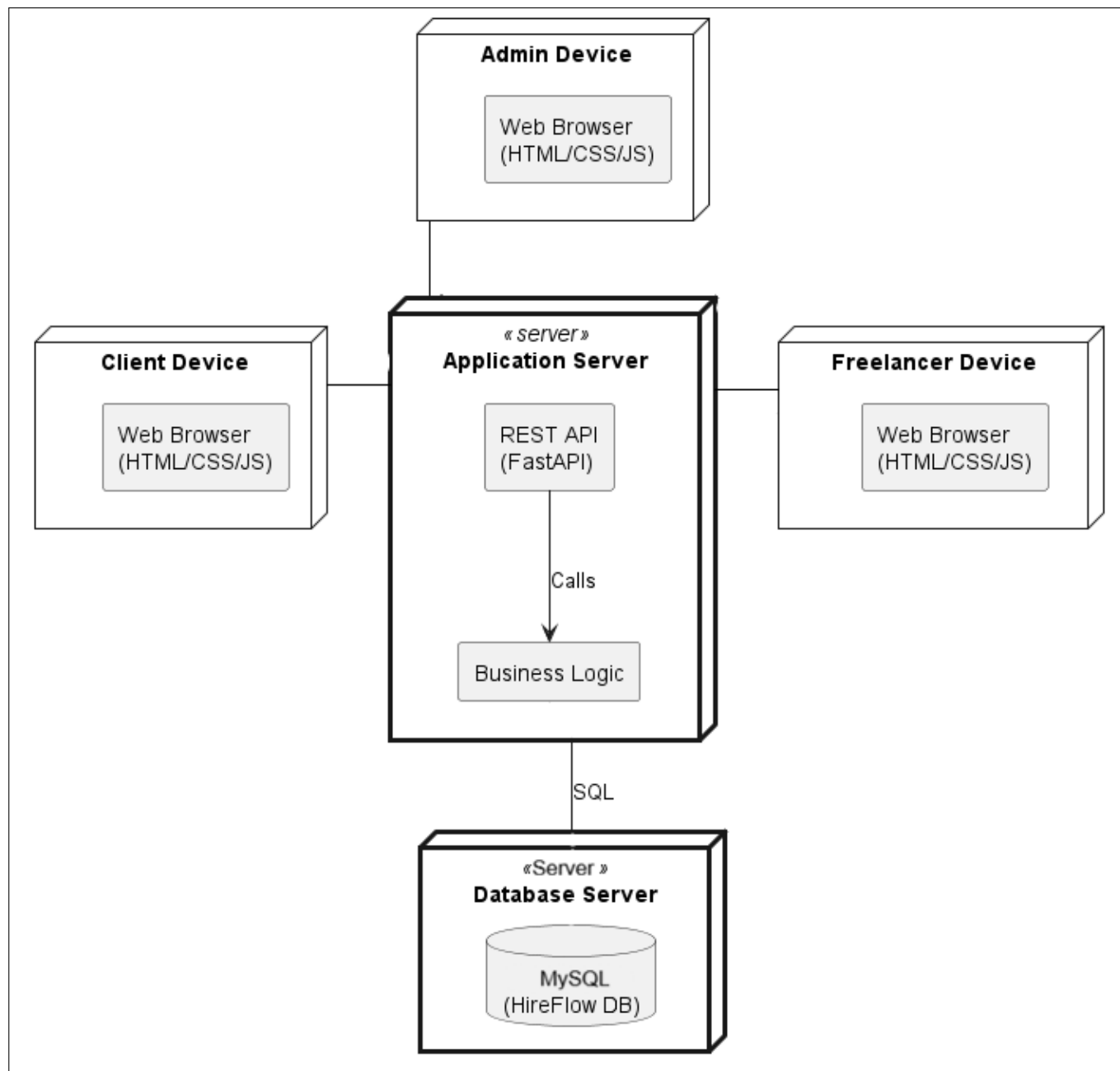


Figure 18: Deployment diagram — HireFlow

The architecture follows a three-tier model:

- **Client tier:** Angular single-page application running in the browser.
- **Application tier:** FastAPI / Python REST API server handling business logic and JWT authentication.
- **Data tier:** MySQL relational database storing users, jobs, and proposals.

2.6 Sprint 0 Implementation

For each use case, we present the corresponding implementation screenshots.

2.6.1 Registration Interface

The registration interface allows any visitor to create a new account by providing their name, email address, password, and selecting a role (Client or Freelancer). The system validates the input before sending the data to the server.

HireFlow

Join the platform

Start your journey as a freelancer or hire talented students.

Client
Post jobs and hire freelancers

Freelancer
Find jobs and build experience

Create Account
Sign up to get started

NAME: Your name

EMAIL: Email

PASSWORD: Password

ROLE: Select role ▼

Create Account

Already have an account? [Login](#)

Figure 19: Registration page

2.6.2 Login Interface

The login interface enables registered users to access the platform by entering their credentials. Upon successful authentication, the user is redirected to their dashboard and granted access to system functionalities based on their role.

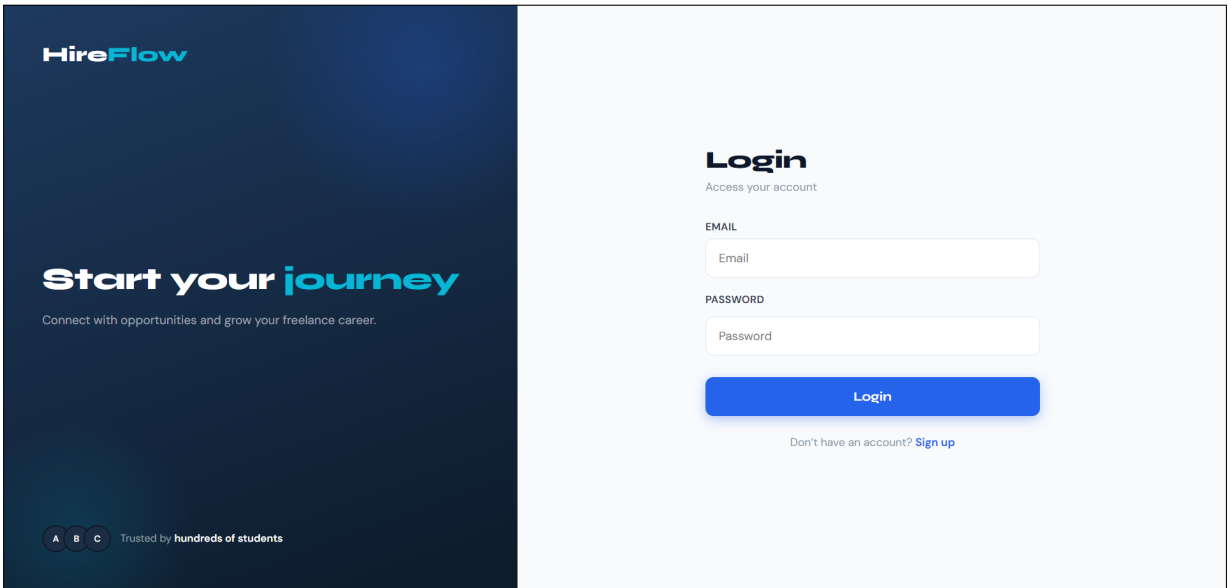


Figure 20: Login page

2.6.3 Manage Users Interface

The user management interface allows administrators to view the list of users, consult user details, and delete users when necessary. The interface provides a clear overview of all registered users and supports basic management operations.

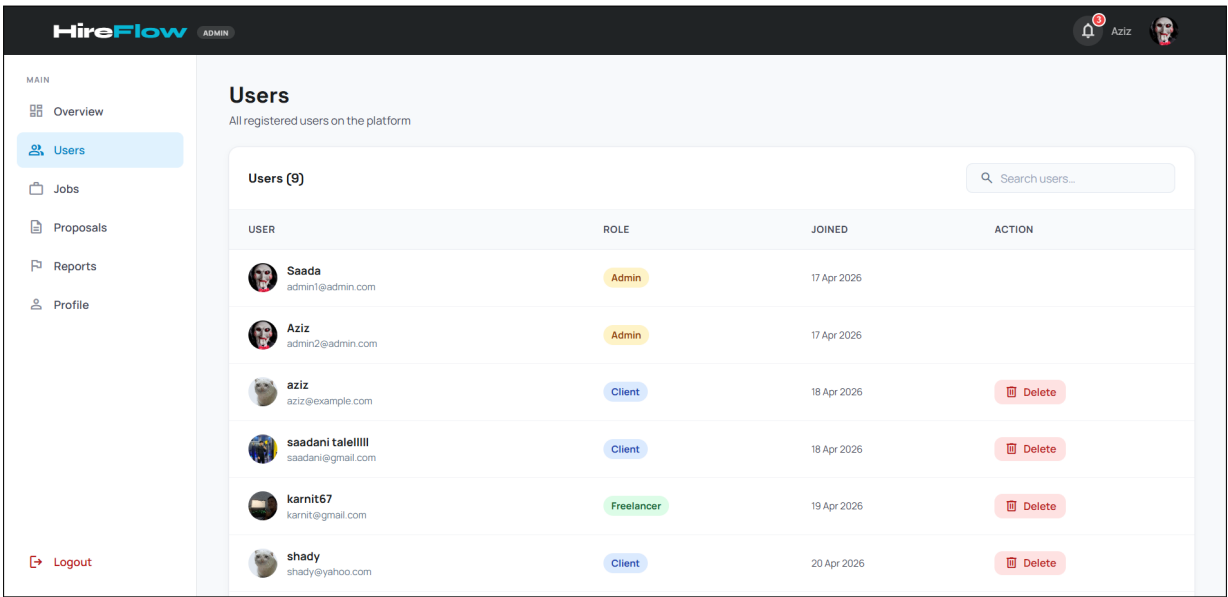


Figure 21: Manage users page

2.6.4 Manage Jobs Interface

The job management interface allows clients to view available jobs, add new jobs by entering required details (such as title and budget), and delete existing jobs. The system ensures validation before performing any operation.

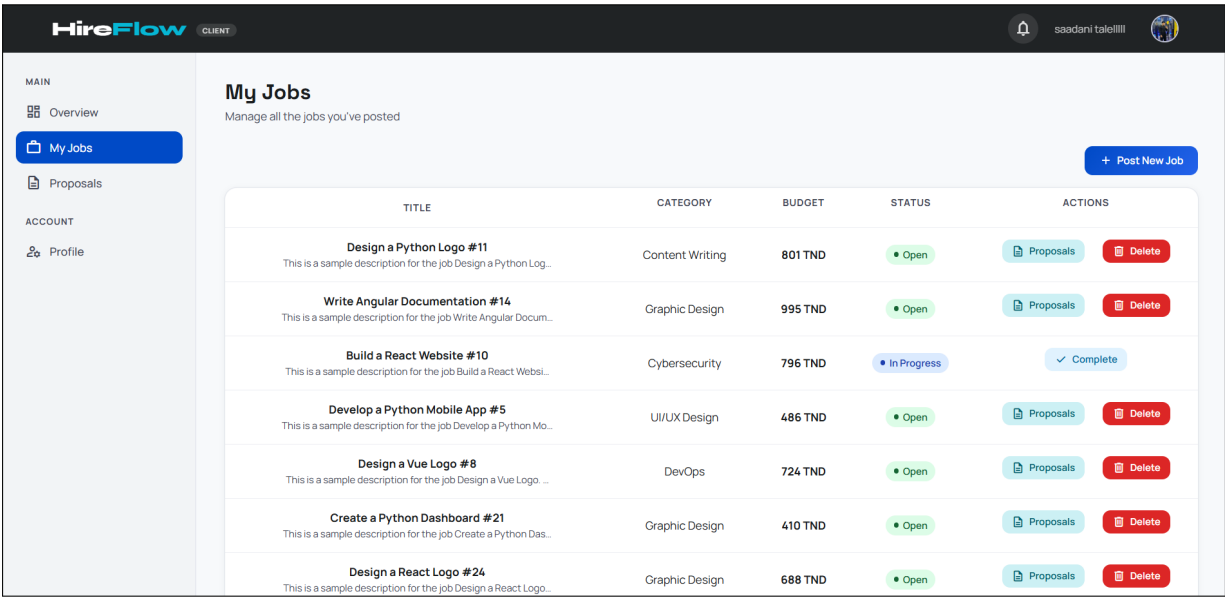


Figure 22: Manage jobs page

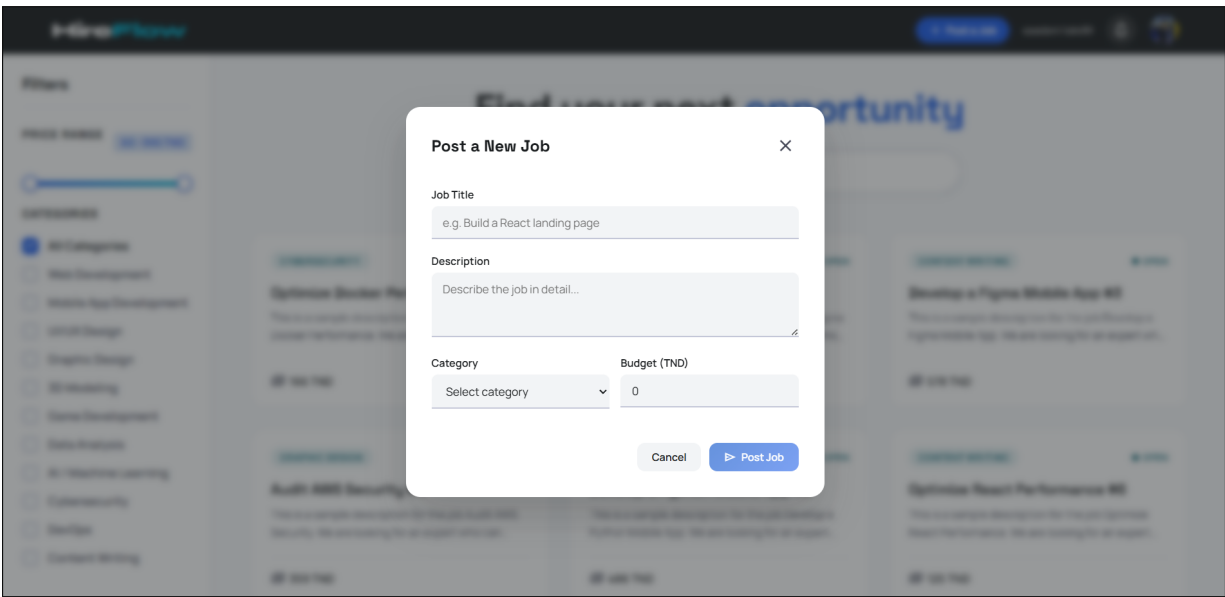


Figure 23: Add job modal

2.6.5 Manage Proposals Interface

The proposal management interface allows freelancers to submit proposals to specific jobs from the jobs section. Additionally, freelancers can access the proposals section to view, update, or delete their existing proposals. The system validates all actions and provides feedback messages after each operation.

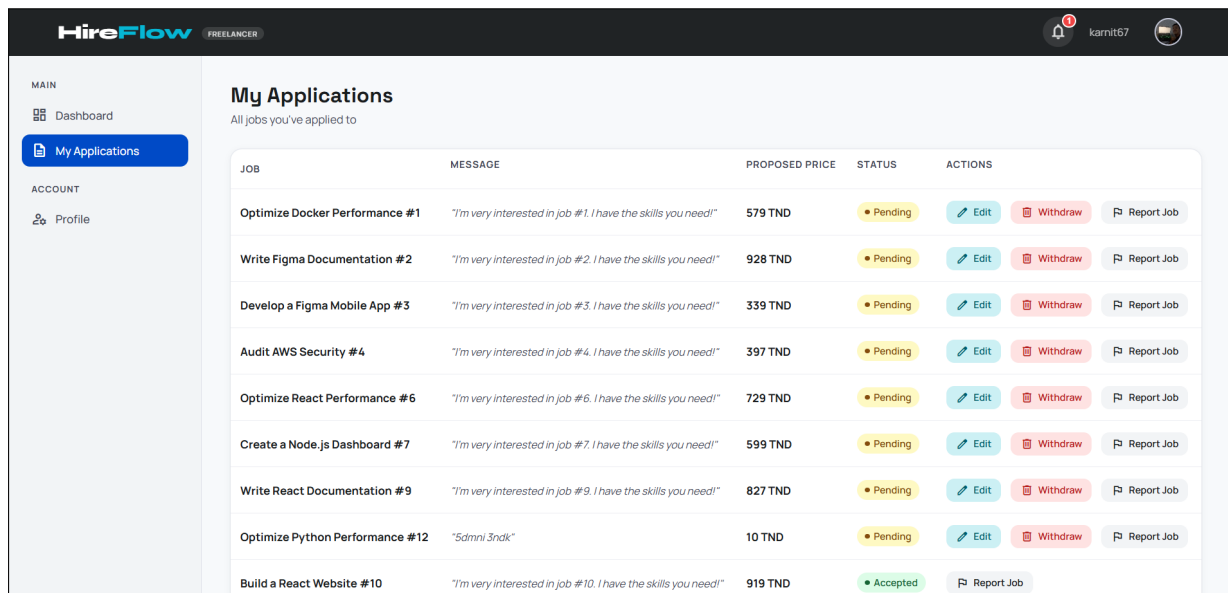


Figure 24: Manage proposals page

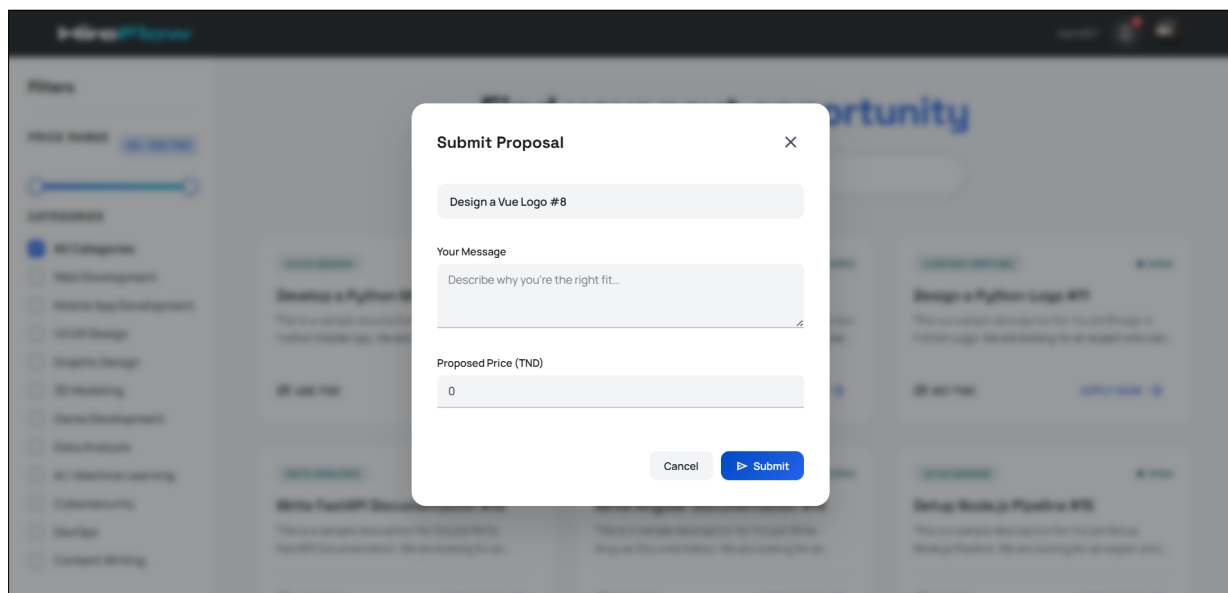


Figure 25: Add proposal modal

2.7 Conclusion

In this chapter, we developed "**Sprint 0**", which focused on the authentication system and the core platform functionalities. We covered the complete sprint life-cycle: **backlog identification**, **refinement** of each use case with textual descriptions, **design** through class and sequence diagrams, and the **implementation** of the corresponding user interfaces.

All planned User Stories "**Register, Log In, Manage Users, Manage Jobs, and Manage Proposals**" were successfully implemented and validated. The system is now **functional and stable**, providing a solid foundation for future development.

In the next chapter, we will develop "**Sprint 1**", which will introduce additional enhancements and extended functionalities to further improve the platform.

3 Sprint 1 – "Core Platform Features"

3.1 Introduction

In this chapter, we present the second phase of the project development, which focuses on the implementation of the platform's core functionalities.

After completing the authentication and basic user access features in "**Sprint 0**", this sprint introduces the main business logic of the freelance platform.

"**Sprint 1**" focuses on user interaction and proposal workflow features. At the end of this sprint, users must be able to handle proposals, edit their profiles, and interact with the system through personalized dashboards.

3.2 Sprint 1 Backlog Identification

The sprint backlog contains all *User Stories* selected from the **Product Backlog** for this sprint. Each User Story is derived directly from the use case diagram and follows the "**INVEST**" criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable).

ID	User Story	Estimation
US06	As a client, I want to handle proposals so that I can accept or reject applications.	High
US07	As a user, I want to view a dashboard so that I can monitor my activity and data.	Medium
US08	As a user, I want to edit my profile so that I can update my personal information.	Medium

Table 10: Sprint 1 Backlog - Core Platform Features

A User Story is considered "**completed**" when:

- The code is developed and reviewed, unit tests pass successfully, the functionality is validated by the Product Owner, and the technical documentation is up to date.

3.3 Sprint 1 Refinement

In this section, we focus on the following use cases:

- Handle proposals;
- Edit Profile;
- View Dashboard.

3.3.1 Refinement of the Use Case “Handle proposals”

The diagram below illustrates the **"Handle proposals"** use case and its interactions with the system actors.

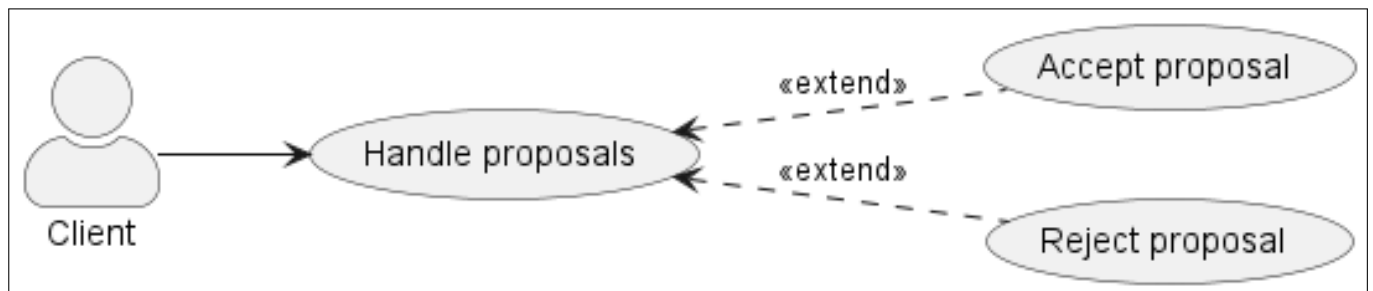


Figure 26: Refinement of the use case “Handle proposals”

Table 11 presents the textual description of the use case **“Handle proposals”**:

Use Case	Handle proposals
Actor	Client
Pre-condition	The client is logged in and has at least one job with received proposals
Post-condition	The selected proposal is accepted or rejected
Main Scenario	1. The client navigates to the proposals section of a specific job. 2. The system displays the list of proposals submitted for that job. 3. The client selects a proposal to review. 4. The system displays the proposal details (freelancer profile, proposal message, proposed budget). 5. The client chooses to accept or reject the proposal. 6. The system validates the action. 7. The system updates the proposal status in the database. 8. The system confirms the successful operation.
Alternative Scenarios	• No proposals available: the system displays a "No proposals yet" message. • Client attempts to accept multiple proposals: the system allows only one accepted proposal per job. • Client cancels the action: the system discards the operation and returns to the proposals list.
Exception	Database connection failure; the system displays an error message.

Table 11: Textual description — Handle proposals

3.3.2 Refinement of the Use Case “Edit Profile”

The diagram below illustrates the "**Edit Profile**" use case and its interactions with the system actors.

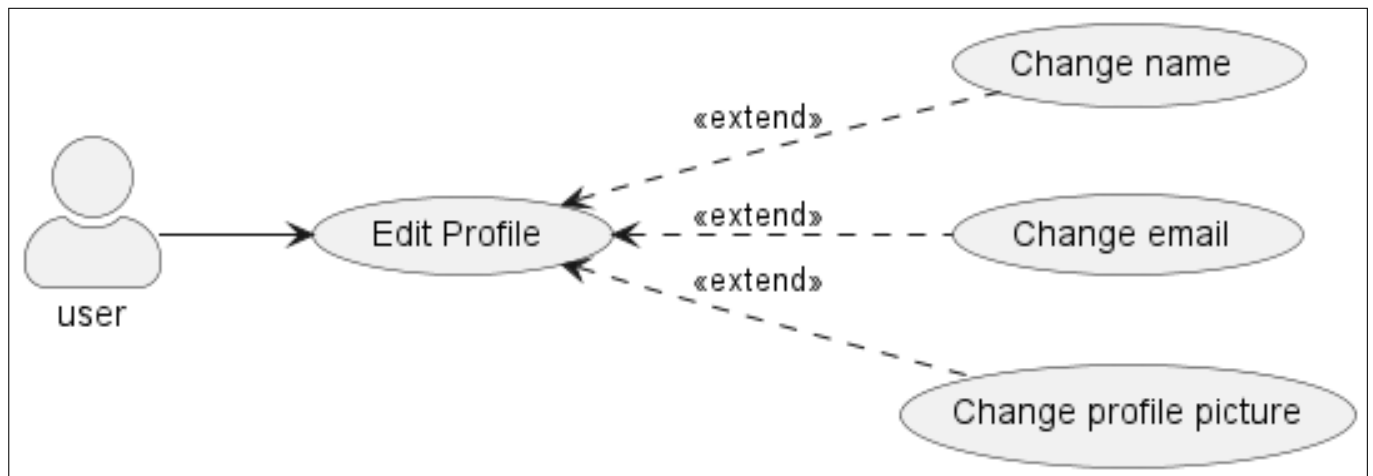


Figure 27: Refinement of the use case “Edit Profile”

Table 12 presents the textual description of the use case “**Edit Profile**”:

Use Case	Edit Profile
Actor	Registered User (Client / Freelancer / Admin)
Pre-condition	The user is logged in to their account
Post-condition	The user’s profile information is updated successfully in the system
Main Scenario	1. The user navigates to the profile settings page. 2. The system displays the current profile information. 3. The user selects the fields to modify (name, email, profile picture). 4. The user makes the desired changes and submits the form. 5. The system validates all input fields. 6. The system updates the user record in the database. 7. The system confirms the successful update and refreshes the profile view.
Alternative Scenarios	• Invalid file format for profile picture: the system displays an error and accepts only JPG, PNG formats. • File size too large: the system rejects upload and shows maximum size limit. • User cancels edit: the system discards changes and returns to view mode. • Validation error: the system highlights invalid fields without saving.
Exception	Database update failure; the system rolls back changes and displays an error message.

Table 12: Textual description — Edit Profile

3.3.3 Refinement of the Use Case “View Dashboard”

The diagram below illustrates the "**View Dashboard**" use case and its interactions with the system actors.

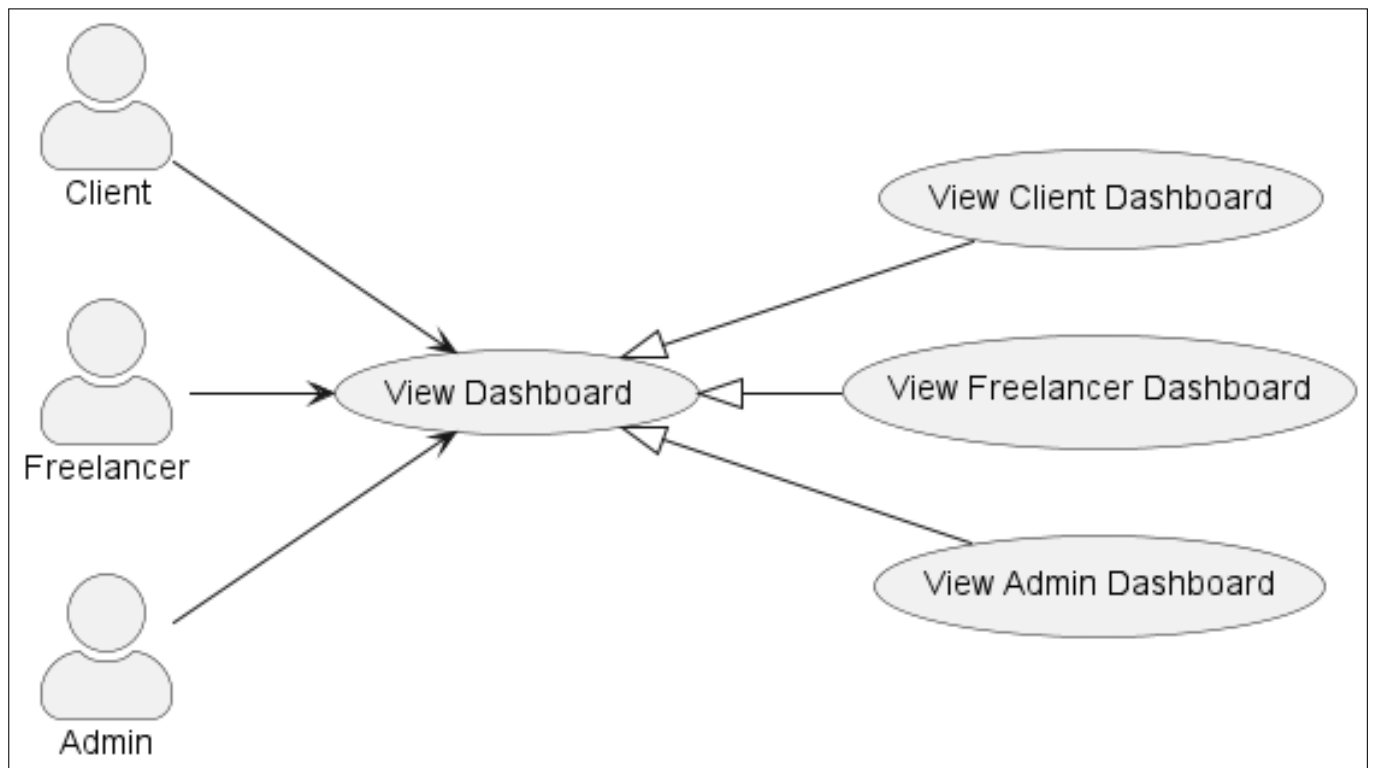


Figure 28: Refinement of the use case “**View Dashboard**”

Table 13 presents the textual description of the use case “**View Dashboard**”:

Use Case	View Dashboard
Actor	Registered User (Client / Freelancer / Admin)
Pre-condition	The user is logged in to their account
Post-condition	The user views personalized statistics and activity summary
Main Scenario	1. The user logs in and is redirected to their dashboard. 2. The system retrieves user-specific data based on their role. 3. For Client role, the dashboard displays: • Number of active jobs • Total proposals received • Jobs in progress 4. For Freelancer role, the dashboard displays: • Active proposals submitted • Accepted/rejected jobs 5. For Administrator role, the dashboard displays: • Total users count • Total jobs count • Platform activity statistics
Alternative Scenarios	• No data available: the system displays "No activity yet" message with suggested actions. • Refresh dashboard: the user can manually refresh to update statistics. • Network delay: the system displays loading indicators while fetching data.
Exception	Data retrieval failure; the system displays cached data if available or an error message.

Table 13: Textual description — View Dashboard

3.4 Sprint 1 Conception

For each Use Case model, we present the Class diagram and the Sequence diagram.

3.4.1 Conception of Use Case “Handle proposals”

a) Class Diagram — Handle proposals

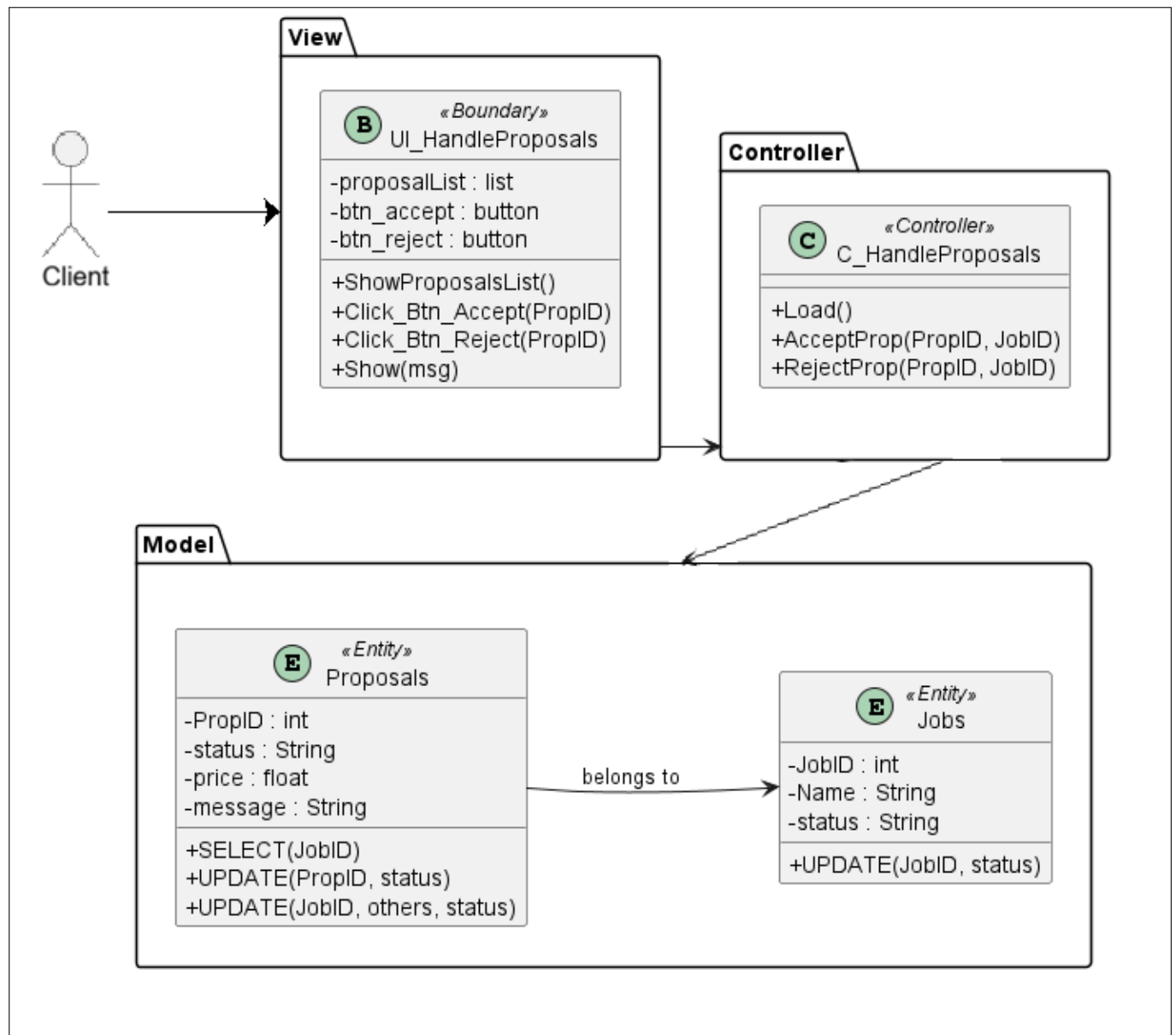


Figure 29: Class diagram — Handle proposals

b) Sequence Diagram — Handle proposals

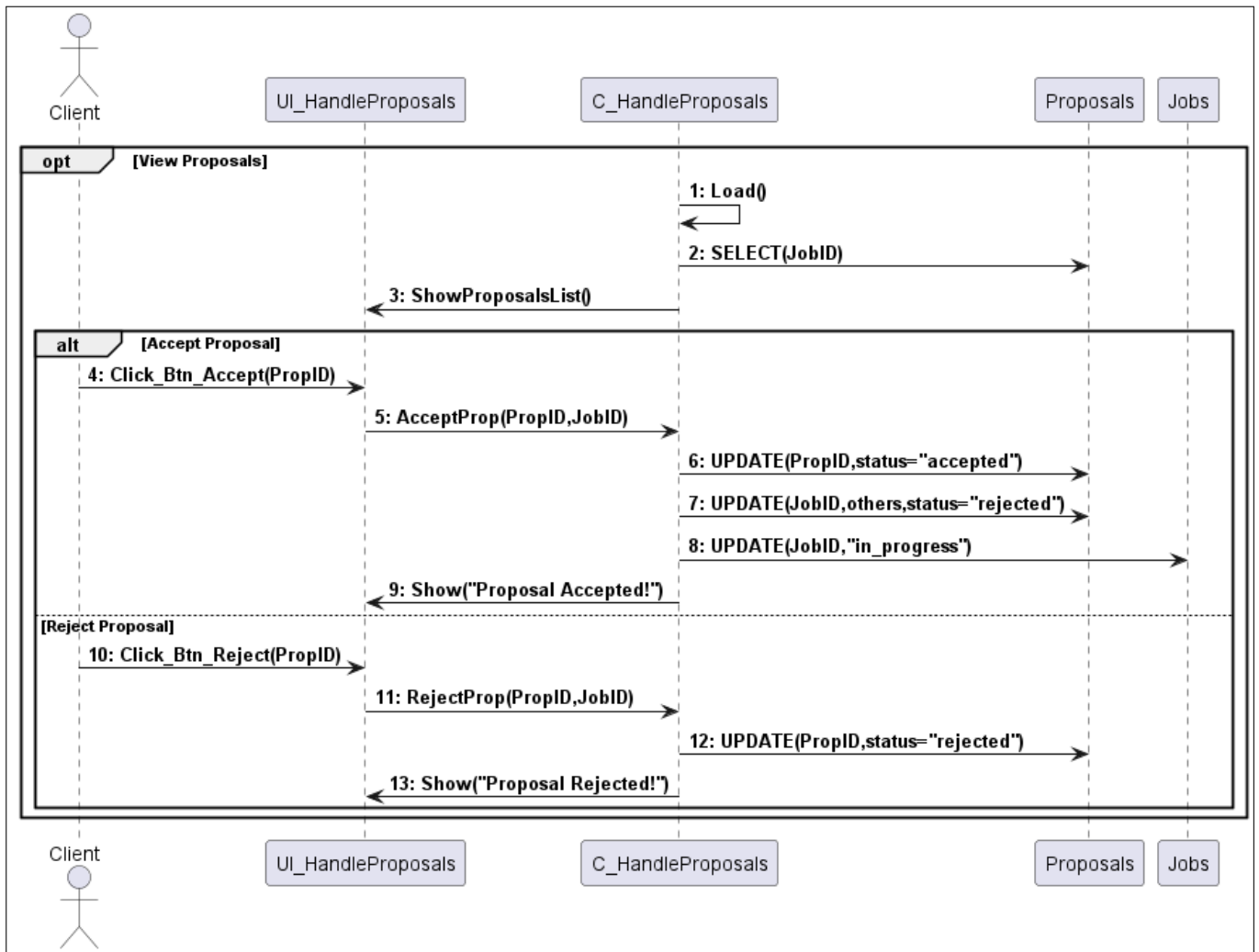


Figure 30: Sequence diagram — Handle proposals

3.4.2 Conception of Use Case “Edit Profile”

a) Class Diagram — Edit Profile

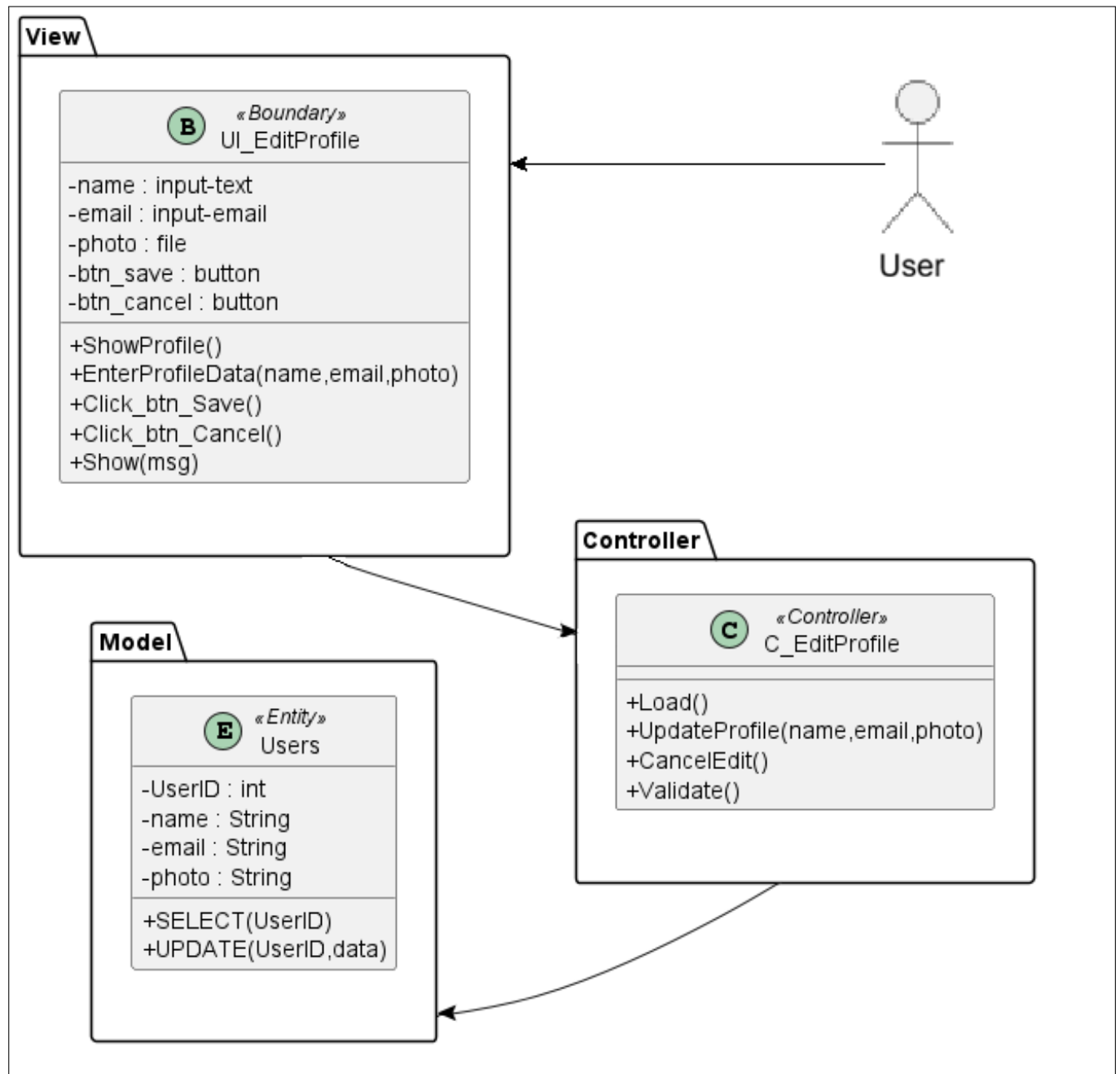


Figure 31: Class diagram — Edit Profile

b) Sequence Diagram — Edit Profile

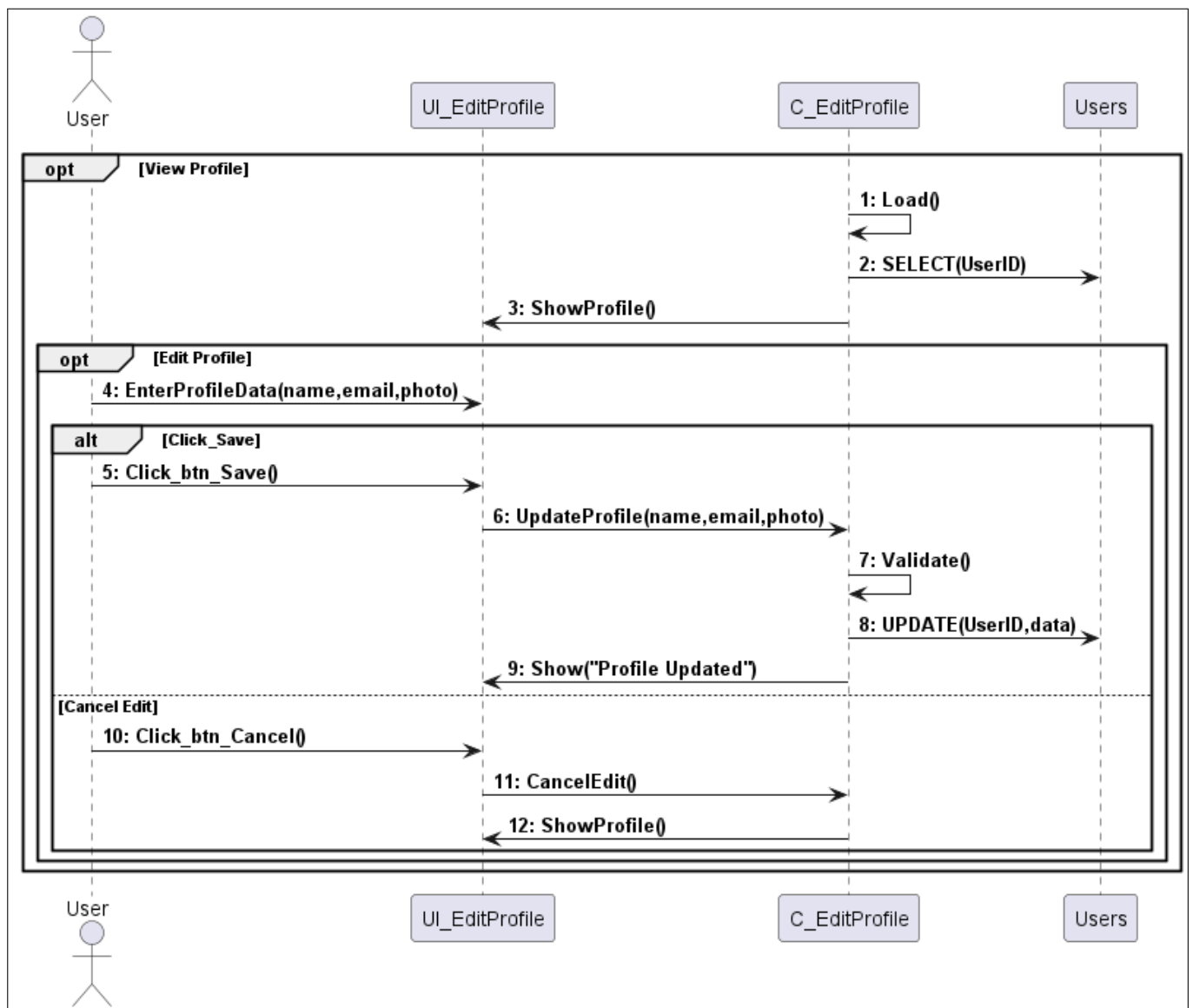


Figure 32: Sequence diagram — Edit Profile

3.4.3 Conception of Use Case “View Dashboard”

a) Class Diagram — View Dashboard

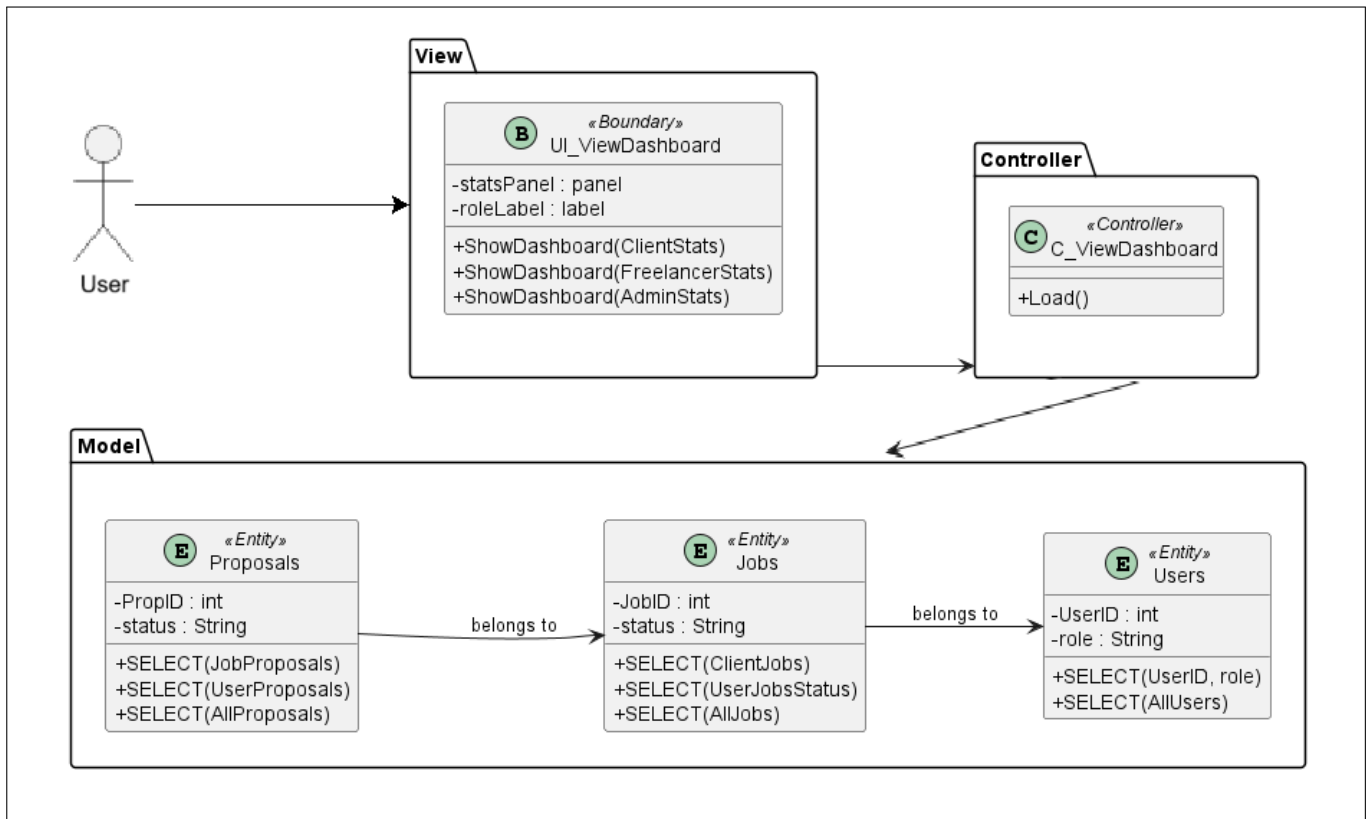


Figure 33: Class diagram — View Dashboard

b) Sequence Diagram — View Dashboard

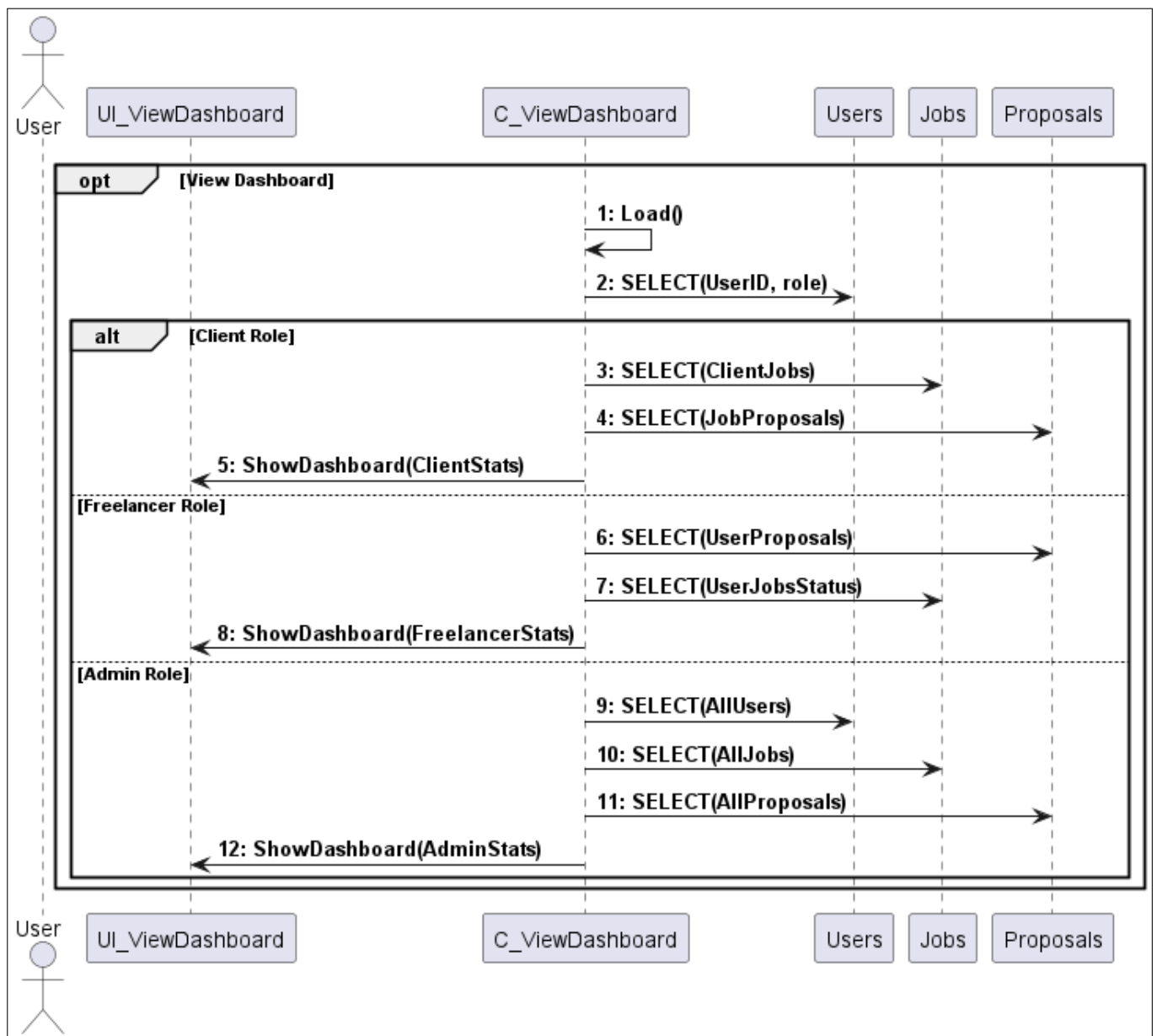


Figure 34: Sequence diagram — View Dashboard

3.5 Sprint 1 Implementation

For each use case, we present the corresponding implementation screenshots.

3.5.1 Handle proposals Interface

The handle proposals interface allows clients to review all proposals submitted for their job postings. For each proposal, the client can consult the freelancer's profile and cover letter, then accept or reject the application. The system updates the proposal status accordingly and confirms the operation.

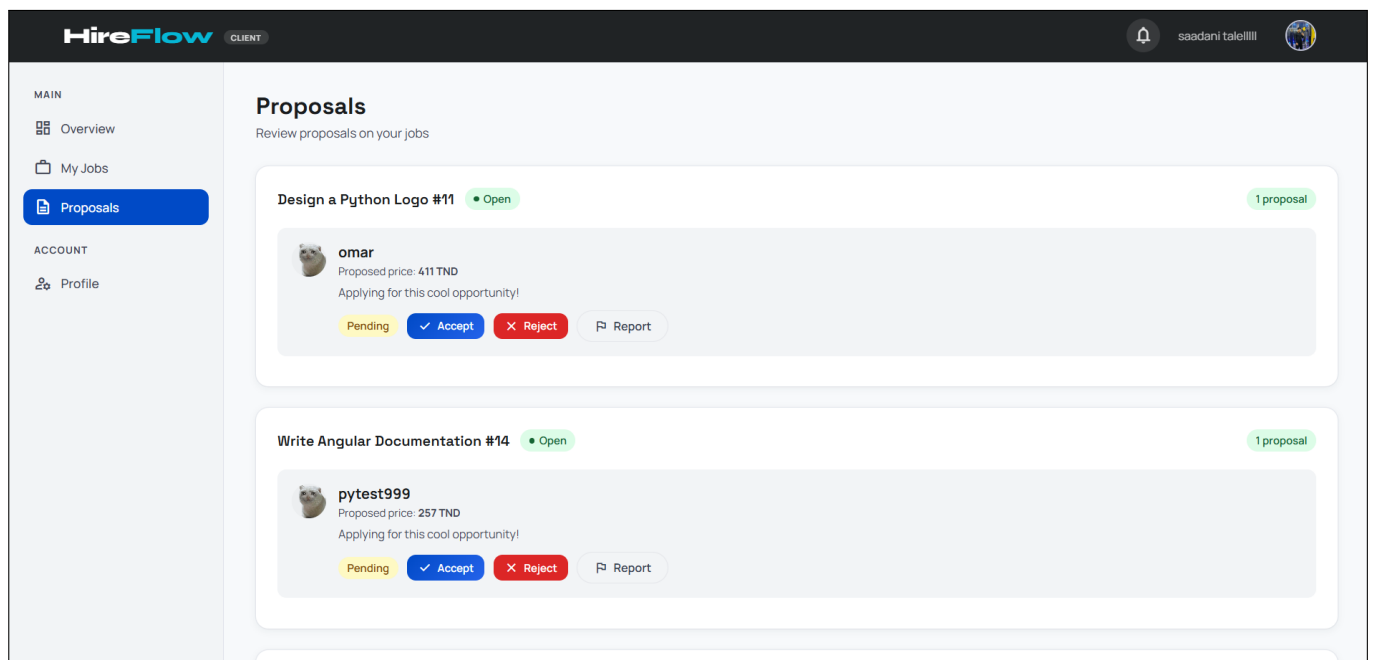


Figure 35: Handle proposals page

3.5.2 Edit Profile Interface

The edit profile interface enables any registered user to update their personal information, including name, email address, password, and profile picture. The system validates the input in real time and confirms the changes upon successful update.

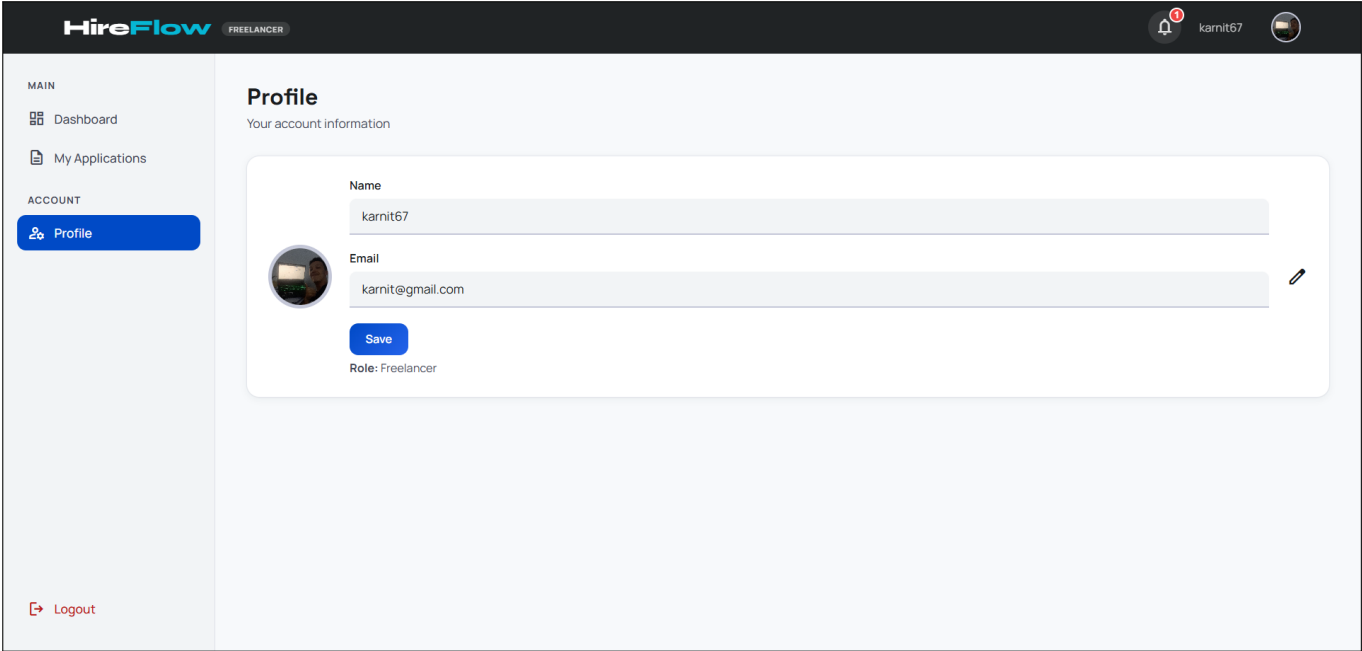


Figure 36: Edit Profile page

3.5.3 View Dashboard Interface

The dashboard interface displays role-specific statistics and activity summaries for each user. Clients see their active jobs and received proposals, freelancers view their submitted proposals and accepted jobs, and administrators monitor platform-wide statistics.

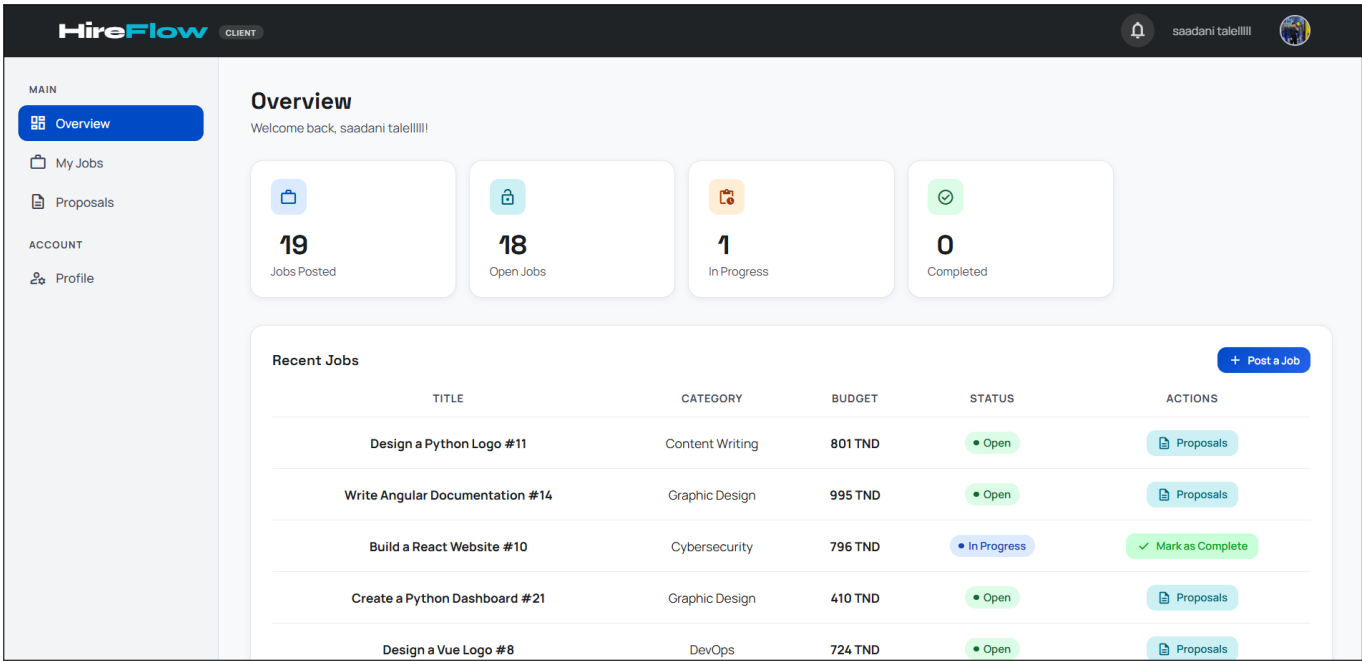


Figure 37: View Dashboard – Client

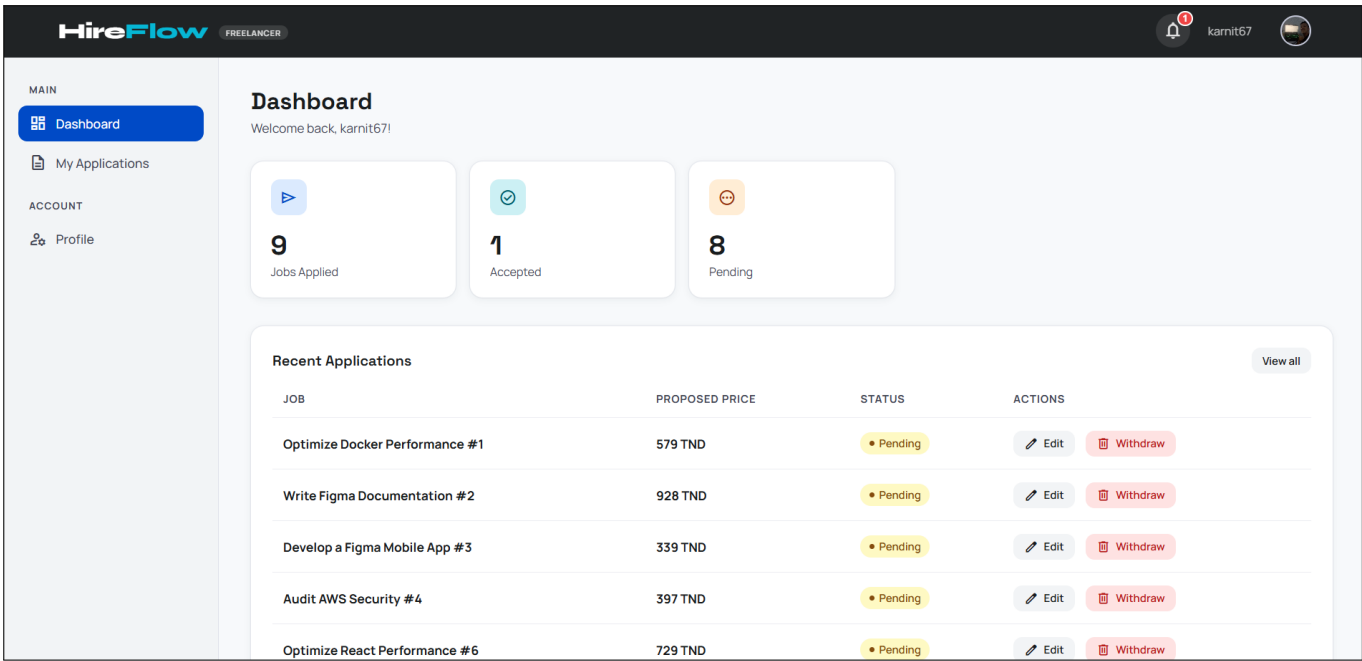


Figure 38: View Dashboard – Freelancer

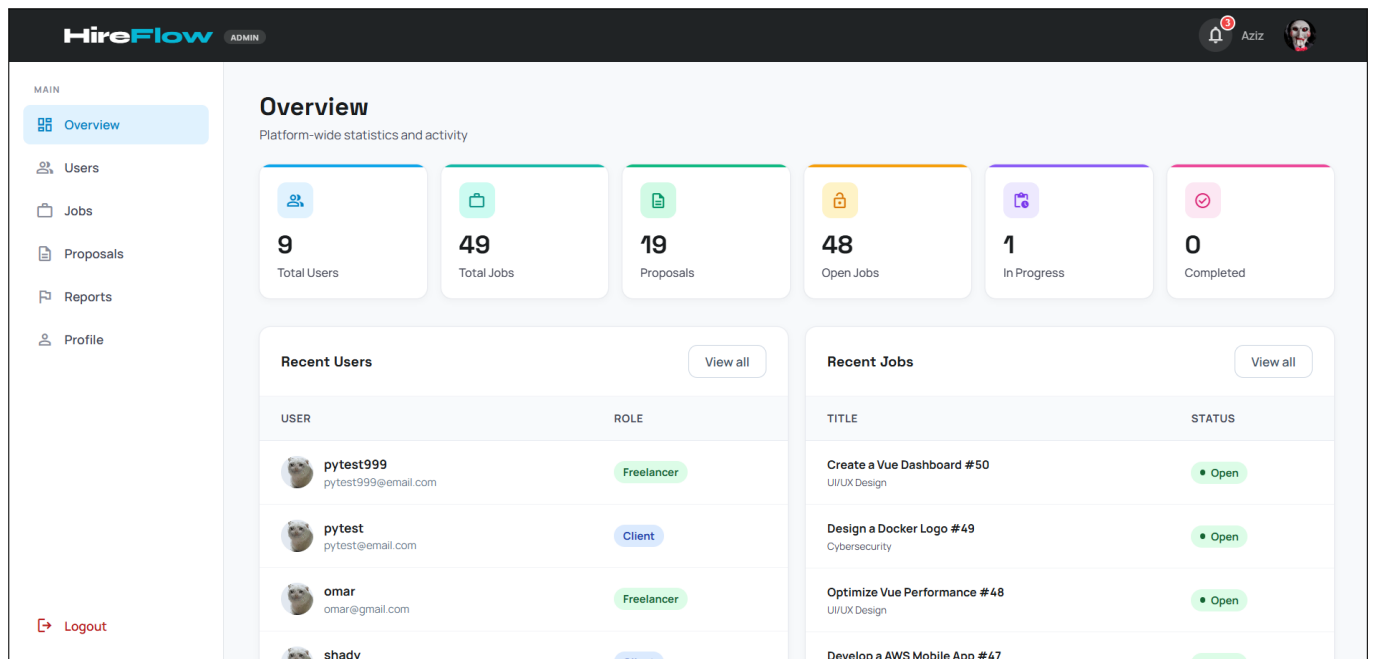


Figure 39: View Dashboard – Administrator

3.6 Conclusion

In this chapter, we developed "**Sprint 1**", which focused on the core platform features. We covered the complete sprint life-cycle: **backlog identification**, **refinement** of each use case with textual descriptions, **design** through class and sequence diagrams, and the **implementation** of the corresponding user interfaces.

All planned User Stories "**Handle proposals, Edit Profile, and View Dashboard**" were successfully implemented and validated. The system now supports full proposal workflow management, personalized dashboards, and profile customization, providing a complete and functional platform experience.

General Conclusion

Our experience developing the **HireFlow** platform over the course of this federated project has allowed us to consolidate our skills in web development, software architecture, database management, and agile project management. Working within an academic context while applying professional methodologies gave us a comprehensive view of requirements analysis, team collaboration, and the application of software engineering best practices.

The HireFlow freelance marketplace was carried out with rigor and commitment, resulting in a functional and well-structured solution adapted to the needs of university students. The primary objective was to develop a performant system facilitating job posting, proposal management, user authentication, and role-based access control between the different actors of the platform.

Chapter 1 established the project framework by presenting the system context, identifying the actors and their interactions, and defining both the functional and non-functional requirements, which allowed us to clearly outline the scope of the solution. Chapter 2 introduced the first sprint, **Sprint 0**, which focused on the authentication system and the core shared features, laying the essential foundations of the platform. Chapter 3 continued the development with **Sprint 1**, enriching the application with proposal handling, profile editing, and personalized dashboards, enhancing the platform's interactivity and usability.

This project has represented an ideal opportunity to put our academic knowledge into practice, while deepening our mastery of project management through the **Scrum** agile methodology. This approach allowed us to effectively organize our work, prioritize tasks, and ensure an iterative and structured progression, fully aligned with the real needs of the platform's users.

As future perspectives, we envision extending the platform with a real-time messaging system between clients and freelancers, an integrated payment module, a rating and review system, and mobile application support, all of which would further enhance the user experience and broaden the reach of HireFlow.

Bibliography

- [1] **SCRUM** : “<https://www.scrum.org/resources/what-scrum-module>” consulted on 07/04/2026
- [2] **The Scrum Guide** : “<https://scrumguides.org/scrum-guide.html>” consulted on 10/04/2026
- [3] **Angular** : “<https://angular.io/>” consulted on 12/04/2026
- [4] **TypeScript** : “<https://www.typescriptlang.org/>” consulted on 15/04/2026
- [5] **FastAPI** : “<https://fastapi.tiangolo.com/>” consulted on 08/04/2026
- [6] **Python** : “<https://www.python.org/>” consulted on 18/04/2026
- [7] **MySQL** : “<https://www.mysql.com/>” consulted on 22/04/2026
- [8] **Visual Studio Code** : “<https://www.blogdumoderateur.com/tools/visual-studio-code/>” consulted on 05/04/2026
- [9] **Git** : “<https://learn.microsoft.com/fr-fr/devops/develop/git/what-is-git>” consulted on 14/04/2026
- [10] **GitHub** : “<https://github.com/>” consulted on 19/04/2026
- [11] **StarUML** : “<https://staruml.io/>” consulted on 25/04/2026
- [12] **PlantUML** : “<https://plantuml.com/>” consulted on 28/04/2026